阶段三至六：Agent 智能体 + Skill 技能 + 斜杠命令与交互 + 工具详解

> 本教程涵盖 Claude Code 的高级交互能力，从 Agent 智能体并行处理到 Skill 技能和工具系统。

目录

- 阶段三至六：Agent 智能体 + Skill 技能 + 斜杠命令与交互 + 工具详解
  - 目录
  - 第一章：Agent 智能体
    - 1.1 什么是 Agent
    - 1.2 Agent 的核心优势
    - 1.3 Agent 类型详解
    - 1.4 Agent 参数配置
    - 1.4.1 Subagent 文件结构与 /agents 命令
    - 1.4.2 调用 Subagent 的方式
    - 1.4.3 前台运行 vs 后台运行
    - 1.4.4 恢复 Subagent 与上下文管理
    - 1.4.5 Hooks 条件规则与 Subagent 生命周期
    - 1.4.6 MCP 服务器限定于 Subagent
    - 1.4.7 禁用特定 Subagent
    - 1.4.8 完整示例：自定义 Subagent 集合
    - 1.5 使用场景与示例
    - 1.5.1 并行编辑 Agents：核心概念与策略
    - 1.5.2 串行编辑 Agents：链式调用与阶段传递
    - 1.5.3 混合模式：并行 + 串行组合拳
    - 1.5.4 并行/串行编辑的监控与结果汇总
    - 1.5.5 避坑指南：编辑 Agents 常见错误
    - 1.5.6 并行 vs 串行编辑 Agents：核心区别
    - 1.6 Agent vs 主对话
    - 1.7 最佳实践
  - 第二章：Skill 技能系统
    - 2.1 什么是 Skill
    - 2.2 Skill 完整列表
    - 2.3 核心 Skill 详解
    - 2.4 自定义 Skill（SKILL.md 格式）
    - 2.5 Skill 场景对照表
    - 2.6 Hooks 系统详解
    - 2.7 MCP 系统详解
    - 2.8 Plugins 插件系统详解
  - 第三章：斜杠命令与交互
    - 3.1 斜杠命令完整列表
    - 3.2 核心命令详解
    - 3.3 快捷键

    - 3.4 对话技巧
    - 3.5 有效的提示词
  - 第四章：工具详解
    - 4.1 Read - 文件读取
    - 4.2 Edit - 文件编辑
    - 4.3 Write - 文件写入
    - 4.4 Glob - 文件搜索
    - 4.5 Grep - 内容搜索
    - 4.6 Bash - 终端执行
  - 实践练习

第一章：Agent 智能体

1.1 什么是 Agent

Agent 是 Claude Code 中的**独立子实例**，可以并行处理复杂任务。每个 Agent 拥有自己的上下文和指令集，能独立执行文件操作、代码编写、终端命令等任务。

```
┌──────────────────────────────────────────────────────────────┐
│                      主对话 (Main Session)                    │
│                                                              │
│    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐  │
│    │  Agent A     │    │  Agent B     │    │  Agent C     │  │
│    │  审查代码    │    │  写测试      │    │  查文档      │  │
│    └──────────────┘    └──────────────┘    └──────────────┘  │
│        ↓                    ↓                    ↓           │
│      并行执行，任务完成后返回结果                              │
└──────────────────────────────────────────────────────────────┘
```

参考, evething in claude

1.2 Agent 的核心优势

特性	说明	实际价值
并行处理	同时启动多个 Agent	效率提升数倍
任务专注	独立上下文，单一任务	避免上下文混淆
独立执行	可后台运行	不阻塞主对话
资源隔离	独立上下文窗口	互不干扰
灵活配置	可配置任务类型和深度	适应不同场景

1.3 Agent 类型详解

Claude Code 提供内置 Agent 类型，Claude 在适当时自动使用。每个都继承父对话的权限，并有额外的工具限制。

主类型（手动调用为主）

Agent 类型	用途	模型	工具	典型场景
general-purpose	通用任务处理	继承主对话	所有工具	默认类型，适用于需要探索和修改的复杂任务
Plan	制定计划和分析	继承主对话	只读工具	复杂项目架构设计、任务分解（plan mode 专用）
Explore	探索性分析	Haiku（快速）	只读工具	代码库调研、需求分析、代码审查

Explore 的详细程度级别：

调用 Explore 时可以指定 `thoroughness` 参数：

级别	说明	适用场景
`quick`	有针对性的快速查找	已知目标，精准搜索
`medium`	平衡的探索	适度调研
`very thorough`	全面深入分析	完整代码库理解

```
Agent:
  instruction: "分析 src/api/ 目录的架构"
  subagent_type: "Explore"
  thoroughness: "medium"  # 可选参数
```

专用 Agent（通常自动调用）

Agent 类型	模型	Claude 何时使用
Bash	继承	在独立上下文中运行终端命令
statusline-setup	Sonnet	当运行 `/statusline` 配置状态栏时
claude-code-guide	Haiku	当提出关于 Claude Code 功能的问题时

选择指南：

```
┌──────────────────────────────────────────────────────────┐
│                        任务类型判断                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  需要深入执行多个步骤？ ──▶ general-purpose              │
│                                                          │
│  需要调研和分析？       ──▶ Explore                      │
│                                                          │
│  需要制定计划？         ──▶ Plan                         │
```

```
│                                                          │
│  学习 Claude Code？    ──▶ claude-code-guide             │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

> **注意：** Task 工具在版本 2.1.63 中已重命名为 Agent。现有代码中的 `Task(...)` 引用仍然作为别名工作。

1.4 Agent 参数配置

调用语法

有两种调用方式：通过工具调用（对话中写配置），以及通过 Subagent 文件（提前定义好）。

工具调用语法：

```
Agent:
  instruction: "你的具体指令"          # 必填：给 Agent 的任务描述
  subagent_type: "general-purpose"      # 必填：Agent 工作模式
  name: "my-agent"                     # 可选：自定义 Agent 名称（用于 @-mention）
  description: "描述"                   # 可选：简短描述（用于后台任务标记）
  model: "sonnet"                      # 可选：模型选择
  maxTurns: 10                         # 可选：最大交互轮数
  run_in_background: false             # 可选：是否后台运行
  tools: [...]                         # 可选：允许的工具列表
  disallowedTools: [...]               # 可选：禁止的工具列表
  permissionMode: "acceptEdits"         # 可选：权限模式
  skills: [...]                        # 可选：预加载的 Skills
  mcpServers: [...]                    # 可选：MCP 服务器
  hooks: {...}                         # 可选：生命周期 Hooks
  memory: "user"                       # 可选：持久内存
  effort: "medium"                     # 可选：努力级别
  isolation: "worktree"                # 可选：隔离模式
  initialPrompt: "..."                 # 可选：自动首轮提示
```

frontmatter 完整字段说明

字段	必填	说明
`name`	✅	唯一标识符，使用小写字母和连字符
`description`	✅	Claude 何时应该委托给此 Agent（自动调度的依据）
`tools`	×	允许使用的工具列表（留空则继承所有工具）
`disallowedTools`	×	禁止使用的工具（从继承列表中删除）
`model`	×	模型：`sonnet` / `opus` / `haiku` / 完整模型 ID / `inherit`（默认）

字段	必填	说明
`permissionMode`	×	权限模式：`default` / `acceptEdits` / `dontAsk` / `bypassPermissions` / `plan`
`maxTurns`	×	Agent 停止前的最大交互轮数
`skills`	×	启动时预加载到上下文的 Skills（注入完整内容，不只是可用调用）
`mcpServers`	×	此 Agent 可用的 MCP 服务器（可以是内联定义或引用已配置的服务器）
`hooks`	×	限定于此 Agent 的生命周期 Hooks
`memory`	×	持久内存范围：`user` / `project` / `local`（跨会话学习）
`background`	×	`true` = 始终以后台任务运行
`effort`	×	努力级别：`low` / `medium` / `high` / `max`（仅 Opus 4.6）
`isolation`	×	`worktree` = 在临时 git worktree 中运行（隔离副本）
`initialPrompt`	×	作为主会话 Agent 时的自动首轮提示

模型解析优先级（从高到低）：

```
1. CLAUDE_CODE_SUBAGENT_MODEL 环境变量（如果设置）
2. 每次调用的 model 参数
3. Agent 定义的 model frontmatter
4. 主对话的模型（默认）
```

工具控制详解

方式一：允许列表（tools）

```yaml
---
name: safe-researcher
description: 只读研究 Agent
tools: Read, Grep, Glob, Bash
---
```

方式二：禁止列表（disallowedTools）

```yaml
---
name: no-writes
description: 禁止写入文件
disallowedTools: Write, Edit
---
```

两者同时设置时： `disallowedTools` 先应用，然后 `tools` 在剩余池中解析。两者都列出的工具会被删除。

限制可生成的 Subagent 类型：

```
---
name: coordinator
description: 协调多个专业 Agent
tools: Agent(worker), Agent(researcher), Read, Bash
---
```

这是允许列表：只能生成 `worker` 和 `researcher` 两种 Subagent。禁止生成其他类型。

权限模式详解

模式	行为说明
`default`	标准权限检查，带确认提示
`acceptEdits`	自动接受文件编辑
`dontAsk`	自动拒绝权限提示（显式允许的工具仍然工作）
`bypassPermissions`	跳过所有权限提示 ⚠ 谨慎使用
`plan`	Plan mode（只读探索）

> **注意：** 如果父级使用 `bypassPermissions`，此设置优先且无法被覆盖。如果父级使用 auto mode，Subagent 继承 auto mode，`permissionMode` frontmatter 被忽略。

持久内存（memory）

`memory` 字段为 Agent 提供一个跨会话持久化的目录，用于积累知识。

范围	存储位置	使用场景
`user`	`~/.claude/agent-memory/<agent-name>/`	在所有项目中共享学习成果
`project`	`.claude/agent-memory/<agent-name>/`	项目特定知识，可版本控制
`local`	`.claude/agent-memory-local/<agent-name>/`	项目特定知识，不检入版本控制

启用内存时：

- Agent 的系统提示自动包含读写内存目录的指令
- 自动加载 `MEMORY.md` 的前 200 行作为上下文（超过 200 行则自动摘要）
- Read、Write、Edit 工具自动启用（用于管理内存文件）

使用建议： 在指令中明确要求 Agent 在开始前查阅内存、完成后保存学习内容。

努力级别（effort）

级别	说明	适用场景
`low`	快速响应，减少探索	简单、明确的任务
`medium`	平衡速度和深度	默认，大多数任务
`high`	更深入的探索和分析	复杂问题
`max`	最大努力（仅 Opus 4.6）	关键决策、深度研究

隔离模式（isolation）

设置为 `worktree` 时，Agent 在临时的 git worktree 中运行，获得仓库的隔离副本。如果 Agent 没有做任何更改，worktree 自动清理。适用于需要大量写操作但不希望影响当前分支的场景。

maxTurns 配置建议

任务复杂度	建议 maxTurns
简单任务（5分钟内）	5-10
中等任务（10-30分钟）	15-20
复杂任务（1小时+）	30-50
超长任务	不设限制

1.4.1 Subagent 文件结构与 /agents 命令

Subagent 使用 YAML frontmatter + Markdown 正文的文件格式定义，存储在指定目录中。

/agents 命令

运行 `/agents` 打开交互式界面，可执行以下操作：

- 查看所有可用 Subagent（内置、用户、项目、plugin）
- 使用引导式设置或 Claude 生成创建新的 Subagent
- 编辑现有 Subagent 的配置和工具访问
- 删除自定义 Subagent
- 查看重复定义时哪些 Subagent 处于活跃状态

非交互式列出所有 Subagent：

```
claude agents
```

显示按来源分组的 Agent 列表，并标注哪些被更高优先级的定义覆盖。

文件存放位置与优先级

位置	范围	优先级	创建方式

位置	范围	优先级	创建方式
`--agents` CLI 标志	当前会话	1（最高）	JSON 传入
`.claude/agents/`	当前项目	2	交互式或手动
`~/.claude/agents/`	所有项目	3	交互式或手动
Plugin 的 `agents/`	启用 plugin 的位置	4（最低）	Plugin 安装

> **注意：** Subagent 在会话启动时加载。如果手动添加文件，需重启会话或运行 `/agents` 立即加载。

Subagent 文件格式

```
---
name: code-reviewer
description: Reviews code for quality and best practices
tools: Read, Grep, Glob, Bash
model: sonnet
---

You are a code reviewer. When invoked, analyze the code and provide
specific, actionable feedback on quality, security, and best practices.
```

- **frontmatter（YAML）**：定义元数据和配置
- **正文（Markdown）**：Agent 的系统提示，指导其行为

Subagent 只接收自己的系统提示（加上基本环境信息，如工作目录），**不继承**主对话的完整系统提示。

CLI 方式创建 Subagent

在启动 Claude Code 时通过 `--agents` 传入 JSON（仅存在于该会话）：

```
claude --agents '{
  "code-reviewer": {
    "description": "Expert code reviewer. Use proactively after code
changes.",
    "prompt": "You are a senior code reviewer. Focus on code quality,
security, and best practices.",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  },
  "debugger": {
    "description": "Debugging specialist for errors and test failures.",
    "prompt": "You are an expert debugger. Analyze errors, identify root
causes, and provide fixes."
  }
}'
```

支持的字段与文件格式相同，用 `prompt` 字段代替 markdown 正文。

1.4.2 调用 Subagent 的方式

有三种模式，从随意建议升级到会话范围默认值：

方式一：自然语言（最随意）

```
Use the test-runner subagent to fix failing tests
Have the code-reviewer subagent look at my recent changes
```

直接命名 Subagent，Claude 会判断是否委托。适合日常对话。

方式二：@-mention（保证指定 Agent 运行）

```
@"code-reviewer (agent)" look at the auth changes
```

输入 `@` 从补全中选择 Subagent。这确保特定 Agent 运行，而不是让 Claude 自行决定。

- 本地 Subagent：`@agent-<name>`
- Plugin Subagent：`@agent-<plugin-name>:<agent-name>`

> **@-mention 控制的是「用哪个 Agent」，不控制「Agent 收到什么提示」。** 完整消息仍然发送给 Claude，它基于你的要求为 Subagent 编写任务提示。

方式三：会话范围 `--agent`（整个会话使用同一 Agent）

```
# 启动会话，以 Subagent 的系统提示作为主会话
claude --agent code-reviewer

# Plugin Subagent
claude --agent <plugin-name>:<agent-name>
```

Subagent 的系统提示完全替换默认 Claude Code 系统提示（如同 `--system-prompt`）。`CLAUDE.md` 和项目内存仍然正常加载。

也可以在 `.claude/settings.json` 中设置为项目默认值：

```
{
  "agent": "code-reviewer"
}
```

CLI 标志优先级高于 settings。

1.4.3 前台运行 vs 后台运行

模式	说明
前台	阻塞主对话直到完成。权限提示和澄清问题传递给你
后台	与主对话并发运行。启动前 Claude Code 会预批准工具权限，完成后自动拒绝未批准内容

Claude 根据任务决定默认模式。你也可以：

- 明确要求："run this in the background"
- 按 **Ctrl+B** 将运行中的任务放到后台

后台 Subagent 的澄清问题处理： 如果后台 Subagent 需要提出澄清问题，该工具调用失败，但 Subagent 继续。如果后台 Subagent 因权限不足而失败，可以启动前台 Subagent 重试（使用交互式提示）。

禁用后台任务： 设置环境变量 `CLAUDE_CODE_DISABLE_BACKGROUND_TASKS=1`。

1.4.4 恢复 Subagent 与上下文管理

每个 Subagent 调用都创建一个全新上下文的实例。恢复 Subagent 时，保留完整对话历史（包括所有工具调用、结果和推理），从停止处继续。

如何恢复

```
Use the code-reviewer subagent to review the authentication module
[Subagent 完成]

Continue that code review and now analyze the authorization logic
[Claude 使用 SendMessage 恢复 Subagent，保留完整上下文]
```

停止的 Subagent 如果收到 `SendMessage`，会自动在后台恢复，无需新的 `Agent` 调用。

Subagent 转录文件

每个 Subagent 的对话保存在 `~/.claude/projects/{project}/{sessionId}/subagents/agent-{agentId}.jsonl`。可以在转录文件中找到 Agent ID，或让 Claude 提供。

转录持久化规则：

- **主对话压缩时**：Subagent 转录不受影响，存储在独立文件中
- **会话持久化**：Subagent 转录在会话中持久化，重启 Claude Code 后仍可恢复
- **自动清理**：根据 `cleanupPeriodDays` 设置（默认 30 天）自动清理

自动压缩

Subagent 支持与主对话相同的自动压缩逻辑。默认在约 95% 容量时触发压缩。可通过环境变量 `CLAUDE_AUTOCOMPACT_PCT_OVERRIDE` 设置更早触发（如设为 `50`）。

1.4.5 Hooks 条件规则与 Subagent 生命周期

Subagent frontmatter 中的 Hooks

直接在 Subagent 文件中定义 Hooks，仅在该 Subagent 活跃时运行，完成后清理。

常见事件：

事件	匹配器输入	触发时机
`PreToolUse`	工具名称	Subagent 使用工具之前
`PostToolUse`	工具名称	Subagent 使用工具之后
`Stop`	—	Subagent 完成时（自动转为 `SubagentStop`）

示例：代码审查 Agent，编辑后自动运行 linter：

```
---
name: code-reviewer
description: 代码审查 + 自动检查
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-command.sh $TOOL_INPUT"
  PostToolUse:
    - matcher: "Edit|Write"
      hooks:
        - type: command
          command: "./scripts/run-linter.sh"
---
```

仅读数据库查询 Agent（完整示例）

通过 `PreToolUse` hook 验证 Bash 命令，只允许 SELECT 查询：

```
---
name: db-reader
description: 执行只读数据库查询
tools: Bash
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---
```

```
You are a database analyst with read-only access.
Execute SELECT queries to answer questions about the data.
```

Hook 验证脚本（`./scripts/validate-readonly-query.sh`）：

```
#!/bin/bash
INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

# 阻止 SQL 写入操作（大小写不敏感）
if echo "$COMMAND" | grep -iE
'\b(INSERT|UPDATE|DELETE|DROP|CREATE|ALTER|TRUNCATE|MERGE)\b' > /dev/null;
then
  echo "Blocked: Write operations not allowed. Use SELECT queries only."
>&2
  exit 2  # 退出码 2 = 阻止操作并返回错误消息
fi

exit 0
```

```
chmod +x ./scripts/validate-readonly-query.sh
```

- **退出码 0**：允许操作
- **退出码 2**：阻止操作，错误消息通过 stderr 返回给 Claude
- **其他退出码**：操作失败，但不阻止

项目级 Hooks（响应 Subagent 生命周期）

在 `settings.json` 中配置，响应主会话中的 Subagent 事件：

```
{
  "hooks": {
    "SubagentStart": [
      {
        "matcher": "db-agent",
        "hooks": [
          { "type": "command", "command": "./scripts/setup-db-
connection.sh" }
        ]
      }
    ],
    "SubagentStop": [
      {
        "hooks": [
          { "type": "command", "command": "./scripts/cleanup-db-
```

```
connection.sh" }
        ]
      }
    ]
  }
}
```

1.4.6 MCP 服务器限定于 Subagent

使用 `mcpServers` 字段将 MCP 服务器限定在特定 Subagent 范围内，对主对话不可见。

```
---
name: browser-tester
description: 使用 Playwright 在真实浏览器中测试功能
mcpServers:
  # 内联定义：仅此 Subagent 可用
  - playwright:
      type: stdio
      command: npx
      args: ["-y", "@playwright/mcp@latest"]
  # 引用名称：复用父会话已配置的服务器
  - github
---

Use the Playwright tools to navigate, screenshot, and interact with pages.
```

优势：

- 将 MCP 服务器工具隔离在 Subagent 上下文内
- 避免 MCP 工具描述消耗主对话的上下文空间
- 每个 Subagent 可以有自己专属的 MCP 工具集

1.4.7 禁用特定 Subagent

在 `settings.json` 的 `permissions.deny` 数组中阻止 Claude 使用特定 Subagent：

```
{
  "permissions": {
    "deny": ["Agent(Explore)", "Agent(my-custom-agent)"]
  }
}
```

也可用 CLI 标志：

```
claude --disallowedTools "Agent(Explore)"
```

对内置和自定义 Subagent 都有效。

1.4.8 完整示例：自定义 Subagent 集合

以下是三个实用的自定义 Subagent，涵盖代码审查、调试、数据分析场景。

代码审查 Agent（只读）

```
---
name: code-reviewer
description: 代码审查专家。主动审查代码质量、安全性和可维护性。在编写或修改代码后立即使用。
tools: Read, Grep, Glob, Bash
model: inherit
---

You are a senior code reviewer ensuring high standards of code quality and security.

When invoked:
1. Run git diff to see recent changes
2. Focus on modified files
3. Begin review immediately

Review checklist:
- Code is clear and readable
- Functions and variables are well-named
- No duplicated code
- Proper error handling
- No exposed secrets or API keys
- Input validation implemented
- Good test coverage
- Performance considerations addressed

Provide feedback organized by priority:
- Critical issues (must fix)
- Warnings (should fix)
- Suggestions (consider improving)

Include specific examples of how to fix issues.
```

调试 Agent（可修改）

```
---
name: debugger
description: 调试专家，处理错误、测试失败和异常行为。遇到任何问题时主动使用。
tools: Read, Edit, Bash, Grep, Glob
```

```
---

You are an expert debugger specializing in root cause analysis.

When invoked:
1. Capture error message and stack trace
2. Identify reproduction steps
3. Isolate the failure location
4. Implement minimal fix
5. Verify solution works

For each issue, provide:
- Root cause explanation
- Evidence supporting the diagnosis
- Specific code fix
- Testing approach
- Prevention recommendations

Focus on fixing the underlying issue, not the symptoms.
```

数据科学家 Agent（领域专用）

```
---
name: data-scientist
description: 数据分析专家，处理 SQL 查询、BigQuery 操作和数据洞察。主动用于数据分析任务和查询。
tools: Bash, Read, Write
model: sonnet
---

You are a data scientist specializing in SQL and BigQuery analysis.

When invoked:
1. Understand the data analysis requirement
2. Write efficient SQL queries
3. Use BigQuery command line tools (bq) when appropriate
4. Analyze and summarize results
5. Present findings clearly

Key practices:
- Write optimized SQL queries with proper filters
- Use appropriate aggregations and joins
- Include comments explaining complex logic
- Format results for readability
- Provide data-driven recommendations

Always ensure queries are efficient and cost-effective.
```

持久内存版代码审查 Agent（可积累经验）

```
---
name: code-reviewer
description: 代码审查专家。审查代码质量、安全性和可维护性。在代码变更后使用。
tools: Read, Grep, Glob, Bash, Edit
model: inherit
memory: project
---

You are a senior code reviewer with knowledge of this project's
conventions.

When invoked:
1. Check your memory (MEMORY.md) for project-specific patterns and
conventions
2. Run git diff to see recent changes
3. Focus on modified files
4. Begin review

When you discover patterns, conventions, or recurring issues, update your
memory:
- Note the pattern and where you found it
- Note project-specific naming conventions
- Note architectural decisions

Provide feedback organized by priority:
- Critical issues (must fix)
- Warnings (should fix)
- Suggestions (consider improving)

After completing the review, save your findings to memory for future
reviews.
```

1.5 使用场景与示例

场景一：并行代码审查

适合场景：需要同时审查多个模块或多个文件

```
Agent:
  instruction: |
    审查 src/api/user.ts 文件，关注以下方面：
    1. 安全性问题（SQL注入、XSS等）
    2. 错误处理是否完善
    3. 输入验证是否充分
    4. 是否有潜在的性能问题
  subagent_type: "Explore"
  description: "审查用户API安全性"
```

场景二：制定项目计划

适合场景：大型功能开发前的整体规划

```
Agent:
  instruction: |
    为这个 React 项目制定测试策略：
    1. 分析项目现有测试结构
    2. 设计单元测试策略（哪些需要测）
    3. 设计集成测试策略
    4. 设计 E2E 测试策略
    5. 推荐测试框架和工具
    6. 给出具体的实施步骤
  subagent_type: "Plan"
  maxTurns: 20
```

场景三：复杂重构任务

适合场景：大规模代码迁移，如 React 迁移、架构重构

```
Agent:
  instruction: |
    将项目中所有 React class 组件迁移到 functional component：
    1. 首先扫描 src/components 目录找出所有 class 组件
    2. 逐一迁移，保持功能完全一致
    3. 确保 state 正确转换为 useState
    4. 确保生命周期方法正确转换
    5. 保持原有的 props 和事件处理
  subagent_type: "general-purpose"
  maxTurns: 50
```

场景四：并行研究多个技术方案

适合场景：技术选型时需要同时调研多个方案

```
# 并行启动 3 个 Agent 研究不同方案
Agent 1 (Redis 方案):
  instruction: "调研 Redis 作为缓存方案：优势、劣势、适用场景、集成方式"
  subagent_type: "Explore"

Agent 2 (Memcached 方案):
  instruction: "调研 Memcached 作为缓存方案：优势、劣势、适用场景、集成方式"
  subagent_type: "Explore"

Agent 3 (本地缓存方案):
  instruction: "调研 Node.js 本地缓存方案（如 node-cache）：优势、劣势、适用场景"
  subagent_type: "Explore"
```

场景五：后台任务

适合场景：需要长时间运行的任务

```
Agent:
  instruction: "运行完整的代码质量分析，包括：复杂度分析、重复代码检测、安全扫描"
  subagent_type: "general-purpose"
  run_in_background: true
  description: "后台代码质量分析"
```

实战案例一：遗留系统现代化改造（完整工作流）

背景： 接手一个 3 年前的 Node.js + Express 项目，需要升级技术栈并清理技术债。

目标： 将项目从 Express 迁移到 Fastify，并补充测试。

完整操作流程：

```
# 第一步：用 Plan Agent 制定迁移计划
Agent:
  instruction: |
    分析当前项目的技术栈和架构：
    1. 扫描 package.json，了解所有依赖及版本
    2. 扫描 src/ 目录，列出所有路由、中间件、控制器
    3. 分析数据库 ORM 使用情况（Sequelize/TypeORM/原生）
    4. 评估测试覆盖率现状
    5. 识别与 Express 强耦合的代码（如 app.use/express.json）
    6. 制定从 Express 迁移到 Fastify 的分阶段计划
    输出：详细迁移路线图，标明每个阶段的风险和工时
  subagent_type: "Plan"
  maxTurns: 30
  description: "制定 Express→Fastify 迁移计划"

# 第二步：迁移阶段一（API 路由）
Agent:
  instruction: |
    将 src/routes/ 下的所有 Express 路由迁移到 Fastify 格式：
    - express.Router() → fastify.register()
    - req/res → fastify 的 request/reply 对象
    - express.json() → fastify.addContentTypeParser
    - 迁移后运行 npm test 确保功能不变
    只修改路由层，不要动业务逻辑
  subagent_type: "general-purpose"
  maxTurns: 40
  description: "迁移 API 路由层"

# 第三步：迁移阶段二（中间件）并行执行
Agent:
  instruction: |
```

```
      将 src/middleware/ 下的所有 Express 中间件迁移到 Fastify 插件：
      - 分析每个中间件的功能
      - 转换为 Fastify addHook 方式（preHandler/auth/onRequest）
      - 确保中间件执行顺序不变
    subagent_type: "general-purpose"
    maxTurns: 30
    description: "迁移中间件层"

  Agent:
    instruction: |
      审查 src/controllers/ 和 src/services/ 目录：
      1. 列出所有文件
      2. 识别对 Express API 的直接引用（如 req.body, res.json）
      3. 替换为 Fastify 的 request.raw / reply.raw 等效写法
      4. 不要改变业务逻辑，只做适配
    subagent_type: "general-purpose"
    maxTurns: 30
    description: "适配业务逻辑层"
```

为什么要拆成多个 Agent?

Agent	职责	独立性
Plan Agent	全局分析 + 规划	需要完整上下文
路由迁移 Agent	纯路由层改造	不依赖其他改动
中间件迁移 Agent	插件层改造	可与路由 Agent 并行
业务逻辑适配 Agent	控制器适配	可与中间件 Agent 并行

实战案例二：Bug 并行调查（3 个 Agent 同时排查）

背景： 生产环境出现用户报告"订单支付后状态未更新"的问题。

问题分析： 这个 bug 可能涉及多个环节——支付网关回调、订单状态机、数据库写入、缓存层同步等。

并行调查方案：

```
# 并行启动 3 个 Agent 各自调查一个可能的根因
Agent 1 (支付回调链路):
  instruction: |
    调查支付回调（webhook）处理链路：
    1. 找到支付回调的入口文件（src/payment/ 或 src/webhooks/）
    2. 追踪从收到回调 → 验证签名 → 更新订单状态的全流程
    3. 检查是否有错误被吞掉（try-catch without rethrow）
    4. 检查回调处理的超时和重试机制
    5. 查看最近的 git log 看是否有相关改动
    输出：可疑代码位置列表 + 根因假设
  subagent_type: "Explore"
  description: "调查支付回调链路"
```

```
Agent 2 (订单状态机):
  instruction: |
    调查订单状态机逻辑：
    1. 找到订单状态定义（src/models/order.ts 或类似）
    2. 分析状态转换规则：pending → paid → completed
    3. 检查状态变更的事务性：是否用了数据库事务？
    4. 检查并发场景：同一订单重复回调时的处理
    5. 查看日志，看最后一次状态变更的时间和触发事件
    输出：状态机缺陷分析 + 可能的问题点
  subagent_type: "Explore"
  description: "审查订单状态机"

Agent 3 (缓存与数据库一致性):
  instruction: |
    调查缓存层对订单状态的影响：
    1. 找到缓存相关代码（Redis 读写逻辑）
    2. 分析：支付完成后，缓存和 DB 的更新顺序
    3. 检查是否存在缓存未及时失效的问题
    4. 检查缓存 key 的命名和过期策略
    5. 查看用户查询订单状态时是从缓存还是 DB 读取
    输出：缓存层问题分析
  subagent_type: "Explore"
  description: "检查缓存一致性"
```

调查完成后，主对话汇总结果：

```
根据 3 个 Agent 的调查结论：

Agent 1 发现：支付回调中 try-catch 吞掉了签名验证失败的错误
Agent 2 发现：订单状态更新没有使用事务，存在竞态条件
Agent 3 发现：缓存 TTL 设置为 24h，导致旧状态被长时间返回

综合根因：支付回调在并发情况下，由于缺少事务保护和缓存未及时失效，
导致部分请求的状态更新被覆盖，且旧状态被缓存返回给用户。

修复方案：[由主对话给出具体代码改动]
```

效率对比：

方式	耗时	覆盖度
单一 Agent 串行调查	~3x 时间	可能遗漏细节
3 个 Agent 并行调查	~1x 时间	全面覆盖每个环节

实战案例三：技术债清理 + 新功能并行开发

背景： 需要在修复技术债的同时，开发一个紧急的新功能。两者互不干扰。

```
# 技术债清理（后台运行，不阻塞主对话）
Agent:
  instruction: |
    执行技术债清理任务：
    1. 扫描 src/ 下所有 console.log，替换为结构化日志（logger.info）
    2. 找出所有 any 类型注解，添加具体类型
    3. 找出超过 50 行的函数，标记需要重构
    4. 统一 src/utils/ 下的工具函数命名风格
    5. 为所有 async 函数补充 try-catch
    不要修改业务逻辑，只做规范化处理
  subagent_type: "general-purpose"
  run_in_background: true
  description: "技术债清理"
  maxTurns: 50

# 新功能开发（主对话直接处理）
# 开发订单导出 Excel 功能...
```

这样主对话可以立即开始处理紧急需求，而技术债清理在后台默默进行，两者互不阻塞。

实战案例四：并行 + 串行混合编辑（完整工作流）

背景： 需要为项目添加国际化（i18n）支持，涉及配置、工具函数、组件三层改动，且各层有依赖关系。

需求分析：

- 改动涉及多个层级和目录
- 有些改动可以并行（相互独立），有些必须串行（有依赖）
- 需要验证每步改动后功能正常

混合工作流设计：

```
第一阶段（并行）：各自处理独立的文件
  ├── Agent 1：处理 src/config/i18n.ts 配置文件
  ├── Agent 2：处理 src/utils/ 下工具函数（t(), formatCurrency() 等）
  └── Agent 3：处理 src/components/ 下的 UI 组件

第二阶段（串行）：验证配置后，再处理业务逻辑层
  └── Agent 4：基于前三个 Agent 的输出，处理 src/services/ 业务层

第三阶段（串行）：验证业务逻辑后，处理页面层
  └── Agent 5：处理 src/pages/ 页面组件

第四阶段（并行）：所有层都完成后，并行做收尾检查
  ├── Agent 6：扫描遗漏的硬编码字符串
  └── Agent 7：运行测试确保无回归
```

完整代码实现：

```
# ========== 第一阶段：并行编辑（3 个 Agent 同时处理不同目录）==========

# Agent 1：配置文件
Agent:
  instruction: |
    为 src/config/ 目录添加 i18n 国际化配置：
    1. 创建 src/config/i18n.ts，包含：
       - 支持的语言列表：['zh-CN', 'en-US', 'ja-JP']
       - 默认语言：zh-CN
       - 语言切换工具函数
       - URL 前缀处理逻辑（/en/xxx, /zh/xxx）
    2. 修改 src/config/index.ts，导出 i18n 配置
    使用 TypeScript strict 模式
  subagent_type: "general-purpose"
  maxTurns: 15
  description: "配置 i18n 基础"

# Agent 2：工具函数
Agent:
  instruction: |
    为 src/utils/ 添加国际化工具函数：
    1. 创建 src/utils/i18n.ts：
       - t(key: string, params?: Record<string, string>): string（翻译函数）
       - setLocale(locale: string): void（切换语言）
       - getLocale(): string（获取当前语言）
       - formatCurrency(amount: number, locale?: string): string
       - formatDate(date: Date, locale?: string): string
    2. 创建 src/locales/ 目录，包含：
       - zh-CN.json（中文翻译）
       - en-US.json（英文翻译）
    3. 工具函数需导入翻译 JSON 文件
  subagent_type: "general-purpose"
  maxTurns: 15
  description: "工具函数国际化"

# Agent 3：UI 组件
Agent:
  instruction: |
    为 src/components/ 下的基础组件添加 i18n 支持：
    1. 修改 Button 组件：支持 title/key 属性
    2. 修改 Modal 组件：header/footer 区域支持国际化 key
    3. 修改 Table 组件：分页文案支持国际化
    4. 修改 Form 组件：placeholder/errorMessage 支持 key
    使用 src/utils/i18n.ts 中的 t() 函数
  subagent_type: "general-purpose"
  maxTurns: 20
  description: "UI 组件国际化"

# ========== 第二阶段：串行编辑（等第一阶段全部完成后再执行）==========

Agent:
  instruction: |
```

```
      基于已完成的 i18n 配置和工具函数，处理业务逻辑层：
      1. 修改 src/services/userService.ts：
         - 用户名显示用 t() 包裹
         - 日期时间显示用 formatDate() 处理
      2. 修改 src/services/orderService.ts：
         - 订单状态文案国际化
         - 金额格式化用 formatCurrency()
      3. 确保所有 Service 正确导入 i18n 工具
      不要改动组件和页面，只处理 services 层
    subagent_type: "general-purpose"
    maxTurns: 15
    description: "业务逻辑层国际化"
    # 注意：此 Agent 等待第一阶段 3 个 Agent 全部完成后才启动

  # ========== 第三阶段：串行编辑（依赖第二阶段）==========

  Agent:
    instruction: |
      基于业务逻辑层已完成的 i18n 适配，处理页面层：
      1. 修改 src/pages/Dashboard.tsx：
         - 页面标题、面包屑文案改为 t('page.dashboard.xxx')
         - 时间范围选择器文案国际化
      2. 修改 src/pages/OrderList.tsx：
         - 表格列名、筛选条件文案
         - 空状态、无权限等提示文案
      3. 在每个页面的 useEffect 中初始化语言设置
    subagent_type: "general-purpose"
    maxTurns: 15
    description: "页面层国际化"
    # 注意：此 Agent 等待第二阶段完成后才启动

  # ========== 第四阶段：并行收尾（所有层都完成后）==========

  Agent:
    instruction: |
      扫描项目中所有可能的硬编码中文字符串：
      1. 扫描 src/ 下所有 .ts/.tsx 文件
      2. 找出所有未使用 t() 的中文字符串（使用正则匹配中文）
      3. 排除：正则表达式内的中文、URL 中的中文、注释中的中文
      4. 列出所有遗漏项的文件和行号
    subagent_type: "Explore"
    description: "扫描遗漏硬编码"

  Agent:
    instruction: |
      运行完整测试套件，确保 i18n 改造无回归：
      1. 运行 npm test
      2. 如果有失败，分析是否与 i18n 相关
      3. 列出所有测试失败，分为：i18n 相关 / 无关
      4. 对 i18n 相关的失败，给出修复建议
    subagent_type: "general-purpose"
    run_in_background: true
    description: "i18n 回归测试"
```

1.5.1 并行编辑 Agents：核心概念与策略

什么是并行编辑 Agents

并行编辑 Agents 是指**同时启动多个 Agent，每个 Agent 独立编辑不同的文件或代码区域**，让多个编辑任务同时进行。

```
时间线
──────────────────────────────────────────────────────────────▶

并行编辑：
Agent A ──────────────▶ [文件 A 修改完成]
Agent B ──────────────▶ [文件 B 修改完成]
Agent C ──────────────▶ [文件 C 修改完成]

vs 串行编辑：

Agent A ──────────────▶ [文件 A 完成]
Agent B ──────────────▶ [文件 B 完成]   (等待 A 完成)
Agent C ──────────────▶ [文件 C 完成]   (等待 B 完成)
```

并行编辑的适用条件

条件	说明	判断方法
文件独立	多个 Agent 修改的文件不重叠	检查文件路径是否互不相交
无共享状态	不修改同一个变量、常量、配置	检查是否有共享的 import
无执行顺序	不需要 A 的输出作为 B 的输入	检查任务间是否有依赖

并行最安全的场景：

- 不同目录的文件（`src/api/` vs `src/utils/` vs `src/components/`）
- 不同功能模块（用户模块 vs 订单模块 vs 支付模块）
- 完全独立的新文件创建

不能并行的场景：

- 多个 Agent 修改同一个文件
- A Agent 定义类型，B Agent 使用该类型（需要串行）
- 修改 shared/config.ts 这种公共文件

并行编辑的启动方式

方式一：单次调用多个 Agent（Claude Code 原生并行）

```
# 在单次消息中同时启动 3 个 Agent，它们会并行执行
Agent 1:
  instruction: "修改 src/utils/format.ts，添加日期格式化函数"
  subagent_type: "general-purpose"

Agent 2:
  instruction: "修改 src/utils/validate.ts，添加邮箱验证函数"
  subagent_type: "general-purpose"

Agent 3:
  instruction: "修改 src/utils/crypto.ts，添加加密解密函数"
  subagent_type: "general-purpose"
```

方式二：通过工具参数隔离任务

```
# 每次启动一个 Agent，但指定不同的文件范围，减少冲突
Agent:
  instruction: "修改 src/api/user.ts，只处理用户模块的 API"
  subagent_type: "general-purpose"
  description: "用户 API 编辑"

Agent:
  instruction: "修改 src/api/order.ts，只处理订单模块的 API"
  subagent_type: "general-purpose"
  description: "订单 API 编辑"
```

并行编辑的冲突类型与避免方法

冲突类型一：文件级冲突

```
# × 错误：两个 Agent 修改同一个文件的不同区域
Agent A: "在 src/utils/index.ts 顶部添加新的导出"
Agent B: "在 src/utils/index.ts 底部添加新的导出"
# 结果：后完成的 Agent 会覆盖前一个的改动（或工具报错）

# ✅ 正确：明确指定不同的文件
Agent A: "在 src/utils/string.ts 中添加字符串工具函数"
Agent B: "在 src/utils/number.ts 中添加数字工具函数"
```

冲突类型二：共享依赖冲突

```
# × 错误：两个 Agent 同时修改共享的类型定义
Agent A: "修改 src/types/api.ts，添加 UserRole 类型"
Agent B: "修改 src/types/api.ts，添加 OrderStatus 类型"
# 结果：两个 Agent 各自读到旧版本，互相覆盖
```

```
# ✅ 正确：统一由一个 Agent 处理共享文件
Agent A: "统一修改 src/types/api.ts，添加 UserRole 和 OrderStatus"
Agent B: "继续处理其他文件，不要动 types/"
```

冲突类型三：导入顺序冲突

```
# × 错误：多个 Agent 同时编辑同一个文件的 import 区
Agent A: "在 src/services/user.ts 的 import 区添加新的 import"
Agent B: "在 src/services/user.ts 的 import 区添加另一个 import"

# ✅ 正确：
# 方案 1：由一个 Agent 统一处理 import 区
Agent: "统一处理 src/services/user.ts 的所有 import 和类型补充"

# 方案 2：不同的 Agent 负责不同的代码区域（需精确划分）
Agent A: "只在 src/services/user.ts 的 class UserService {} 内部添加方法"
Agent B: "在 src/services/user.ts 的 import 区添加类型导入（如果必要的话）"
```

1.5.2 串行编辑 Agents：链式调用与阶段传递

什么是串行编辑 Agents

串行编辑 Agents 是指**下一个 Agent 的任务依赖上一个 Agent 的输出结果**，按顺序一个接一个执行。

```
Agent 1 ──▶ 输出结果 ──▶ Agent 2 ──▶ 输出结果 ──▶ Agent 3 ──▶ 最终完成
```

串行编辑的典型场景

场景	原因	示例
依赖分析结果	第二个任务需要知道第一个任务的发现	扫描 → 修复
需要先建立基础	公共文件/类型必须先定义	类型定义 → 使用类型
分阶段验证	每阶段完成后验证，再决定下一步	迁移 → 测试 → 审查
代码生成后需要处理	先生成，再补充关联内容	生成 API → 生成类型 → 生成 Mock

串行编辑的核心技巧：上下文传递

串行 Agent 之间传递信息有三种方式：

方式一：主对话汇总后传给下一个 Agent（推荐）

```
# Agent 1：分析阶段
Agent:
```

```
    instruction: |
      分析 src/legacy/ 目录下的所有文件：
      1. 列出所有文件和主要功能
      2. 识别出需要迁移的 API endpoint 列表
      3. 识别出共享的 utility 函数
      4. 识别出硬编码的配置常量
      输出格式：
      - API endpoints: [列表]
      - Utilities: [列表]
      - Config: [列表]
    subagent_type: "Explore"
    description: "分析待迁移代码"

  # 主对话收到结果后，总结并启动 Agent 2
  # 主对话："根据 Agent 1 的分析，现在启动 Agent 2 逐个迁移..."
  # （主对话作为中间人，将 Agent 1 的输出传递给 Agent 2）

  # Agent 2：迁移阶段
  Agent:
    instruction: |
      基于分析结果，迁移 src/legacy/ 中的 API endpoints 到 src/api/：
      [将 Agent 1 的输出粘贴到这里]
      按照以下优先级迁移：
      1. 先迁移共享 utility 函数（因为 API 依赖它们）
      2. 再迁移不依赖其他 API 的独立 endpoints
      3. 最后迁移复杂依赖的 endpoints
      每次迁移后运行 npm test 验证
    subagent_type: "general-purpose"
    maxTurns: 30
    description: "迁移 API endpoints"
```

方式二：让 Agent 输出可执行的任务清单

```
  # Agent 1：生成任务清单
  Agent:
    instruction: |
      扫描 src/components/ 目录，找出所有需要添加 loading 状态的组件。
      为每个组件生成一条具体的修改指令，格式如下：
      FILE: src/components/Button.tsx
      CHANGE: 在 Button 组件的 props 中添加 loading?: boolean
      CHANGE: 当 loading=true 时，显示 spinner 并禁用按钮
      ...
      输出一个完整的任务清单 JSON（可以放在响应末尾供提取）
    subagent_type: "Explore"
    description: "生成修改任务清单"

  # 主对话提取清单，启动 Agent 2 执行
  # Agent 2: 按清单逐项修改（可以是并行也可以是串行）
  Agent:
    instruction: |
      按以下清单为组件添加 loading 状态支持：
      [粘贴 Agent 1 生成的任务清单]
```

```
    修改规则：
    - 每次修改前后对比，确保功能不变
    - 保持原有 props 接口兼容（loading 是可选的）
    - 统一使用项目中已有的 Spinner 组件
  subagent_type: "general-purpose"
  maxTurns: 25
  description: "批量添加 loading 状态"
```

方式三：使用 TaskOutput 工具获取前序 Agent 结果

```
# Agent 1：标记为后台任务，记录输出
Agent:
  instruction: "生成 API 文档到 src/docs/api.md，涵盖所有 endpoint"
  subagent_type: "general-purpose"
  description: "生成 API 文档"
  run_in_background: true

# 主对话在需要时获取输出
# [等 Agent 1 完成后]

# Agent 2：读取 Agent 1 的输出，继续处理
Agent:
  instruction: |
    读取 src/docs/api.md，基于文档内容：
    1. 为每个 endpoint 生成 curl 示例命令
    2. 添加请求/响应示例 JSON
    3. 补充错误码说明
  subagent_type: "general-purpose"
  description: "补充 API 文档示例"
```

1.5.3 混合模式：并行 + 串行组合拳

为什么需要混合模式

实际项目中，大部分复杂任务既包含**相互独立的部分**（适合并行），也包含**有依赖关系的部分**（必须串行）。

```
项目重构工作流：

Phase 1（并行）：各自分析不同模块
  ├── 分析模块 A ──┐
  ├── 分析模块 B ──┼──▶ 汇总分析报告
  └── 分析模块 C ──┘

Phase 2（串行）：基于分析结果，按依赖顺序迁移
  ├── 迁移共享工具 ────▶
  ├── 迁移模块 A ────▶
  ├── 迁移模块 B ────▶
  └── 迁移模块 C ────▶
```

```
Phase 3（并行）：所有模块迁移完成后，并行验证
  ├── 运行单元测试 ──┐
  ├── 检查类型错误 ──┼──▶ 汇总验证报告
  └── 检查覆盖率 ──┘
```

混合模式实战案例：重构电商后台

项目： Express + Sequelize 电商后台，需要重构为 Fastify + Prisma。

工作流设计：

```
# ═══════════════════════════════════════════════════════════
# 第一阶段：并行探索（分析现有架构）
# ═══════════════════════════════════════════════════════════

Agent:
  instruction: |
    分析 src/routes/ 目录：
    1. 列出所有路由文件和 endpoint
    2. 绘制路由映射图（哪些 URL 对应哪些文件）
    3. 识别路由间的依赖关系
    4. 列出需要迁移的中间件
  subagent_type: "Explore"
  description: "分析路由层"

Agent:
  instruction: |
    分析 src/models/ 目录（Sequelize 模型）：
    1. 列出所有模型及其字段
    2. 分析模型间的关系（hasMany/belongsTo/ManyToMany）
    3. 识别所有关联查询（N+1 问题检查）
    4. 列出需要迁移的 hooks 和 validations
  subagent_type: "Explore"
  description: "分析数据模型"

Agent:
  instruction: |
    分析 src/services/ 和 src/controllers/ 目录：
    1. 列出所有业务逻辑文件
    2. 分析每个 service 使用的 Sequelize API
    3. 识别事务使用情况
    4. 列出需要迁移的业务规则
  subagent_type: "Explore"
  description: "分析业务逻辑层"

# ═══════════════════════════════════════════════════════════
# 第二阶段：串行迁移（按依赖顺序执行）
# ═══════════════════════════════════════════════════════════

# 第二阶段 Step 1：先迁移数据模型（所有层的地基）
```

```
  Agent:
    instruction: |
      基于分析结果，将 Sequelize 模型迁移到 Prisma schema：
      1. 创建 prisma/schema.prisma
      2. 将每个 Sequelize 模型转换为 Prisma model
      3. 转换关联关系（hasMany → relation, belongsTo → @relation）
      4. 转换字段类型（STRING → String, INTEGER → Int, DATE → DateTime）
      5. 转换验证规则（isEmail/isNumeric → @规则）
      6. 转换 hooks 为 Prisma Middleware 格式
      迁移后运行 npx prisma validate 验证语法
    subagent_type: "general-purpose"
    maxTurns: 40
    description: "迁移数据模型"

  # 第二阶段 Step 2：迁移工具函数（不依赖 ORM）
  Agent:
    instruction: |
      迁移 src/utils/ 下的工具函数：
      1. 纯工具函数（date, string, number）→ 保持不变
      2. 涉及 Sequelize 的工具 → 改为 Prisma client 调用
      3. 确保不依赖任何 Sequelize 特有类型
      迁移后确保 npm run build 不报错
    subagent_type: "general-purpose"
    maxTurns: 20
    description: "迁移工具函数"

  # 第二阶段 Step 3：迁移业务逻辑层（依赖 Prisma schema 和工具）
  Agent:
    instruction: |
      迁移 src/services/ 下的业务逻辑：
      1. Sequelize.transaction() → Prisma.$transaction()
      2. Model.findAll() → prisma.model.findMany()
      3. Model.create() → prisma.model.create()
      4. Model.update() → prisma.model.update()
      5. Model.destroy() → prisma.model.delete()
      6. 关联查询：include 替换 eager loading
      7. Op.and / Op.or → AND / OR（Prisma where 语法）
      每次修改后运行相关单元测试
    subagent_type: "general-purpose"
    maxTurns: 40
    description: "迁移业务逻辑层"

  # 第二阶段 Step 4：迁移控制器和路由（依赖 services 层）
  Agent:
    instruction: |
      迁移 src/controllers/ 和 src/routes/：
      1. req.body/req.query → fastify 的 request.body/request.query
      2. res.json() → reply.send()
      3. res.status(404) → reply.code(404).send()
      4. express router → fastify routes（register）
      5. 中间件 → fastify addHook（onRequest/preHandler）
      6. 错误处理中间件 → fastify setErrorHandler
      路由层只改适配器，不改业务逻辑
    subagent_type: "general-purpose"
```

```
    maxTurns: 35
    description: "迁移控制器和路由"

  # ═══════════════════════════════════════════════════════════
  # 第三阶段：并行验证（迁移完成后全面检查）
  # ═══════════════════════════════════════════════════════════

  Agent:
    instruction: |
      运行所有测试，验证重构完整性：
      1. npm test：运行单元和集成测试
      2. 逐一分析失败的测试，判断原因：
         - Sequelize API 残留 → 修复
         - 类型不匹配 → 补充类型定义
         - 业务逻辑问题 → 标记待审查
      3. 报告：哪些通过，哪些失败，失败原因分类
    subagent_type: "general-purpose"
    maxTurns: 30
    description: "验证重构完整性"

  Agent:
    instruction: |
      检查 Prisma 迁移中的性能问题：
      1. 搜索所有 findMany/-findOne 查询
      2. 检查是否遗漏了必要的 include（防止 N+1）
      3. 检查是否有大结果集未做分页（skip/take）
      4. 检查索引使用情况
      5. 输出潜在性能问题清单
    subagent_type: "Explore"
    description: "检查性能问题"

  Agent:
    instruction: |
      审查 src/ 目录下是否还有 Sequelize 残留：
      1. 搜索 'Sequelize'、'Model'、'findAll'、'findOne' 等关键字
      2. 搜索 '@types/sequelize'
      3. 搜索 'package.json' 中是否还有 sequelize 依赖（应保留但不使用）
      4. 输出所有残留引用，标注位置和严重程度
    subagent_type: "Explore"
    description: "检查残留依赖"
```

混合模式决策树

```
发起任务前，先问自己三个问题：

┌──────────────────────────────────────────┐
│  问题 1：这些 Agent 修改的文件有重叠吗？   │
├──────────────────────────────────────────┤
│  有重叠 ──▶ 不能并行，必须串行             │
│  无重叠 ──▶ 继续问题 2                     │
└──────────────────────────────────────────┘
```

```
                    ↓
┌──────────────────────────────────────────┐
│  问题 2：下一个 Agent 需要前一个的输出？  │
├──────────────────────────────────────────┤
│  需要 ──▶ 必须串行（用主对话传递上下文）  │
│  不需要 ──▶ 继续问题 3                    │
└──────────────────────────────────────────┘
                    ↓
┌──────────────────────────────────────────┐
│  问题 3：任务之间是否相互独立？           │
├──────────────────────────────────────────┤
│  独立 ──▶ ✅ 可以并行                     │
│  不独立 ──▶ 需要拆分到独立后并行          │
└──────────────────────────────────────────┘
```

1.5.4 并行/串行编辑的监控与结果汇总

并行 Agent 的监控

启动时的状态：

```
Agent 1（路由迁移）    ──▶ 运行中...
Agent 2（中间件迁移）  ──▶ 运行中...
Agent 3（控制器适配）  ──▶ 等待中...（因为 Agent 1 和 2 还未完成）
```

实际行为： Claude Code 中，所有在同一条消息里启动的 Agent 会并行执行。 但如果后面的 Agent 依赖前面 Agent 的输出，需要显式等待。

等待并行 Agent 完成的方式：

```
# × 错误：直接启动后续 Agent，假设前序已完成
Agent: "开始迁移 controllers（依赖 routes 的改动）"
# Agent 会读取旧版本的代码，导致冲突

# ✅ 正确：等前序 Agent 全部完成后，再启动
# 先确认所有前置 Agent 报告完成，然后在新的消息中启动
Agent: "现在开始迁移 controllers..."
```

Claude Code 的实际处理：

- 在**同一条消息**中启动的多个 Agent **并行执行**
- 后续消息中的 Agent 在收到前序结果后**才启动**
- `run_in_background: true` 的 Agent **不阻塞**主对话

结果汇总的最佳实践

```
# 在所有 Agent 完成后，主对话统一汇总：

"根据各个 Agent 的工作结果，以下是迁移总结：

## 已完成的改动
- prisma/schema.prisma：8 个模型全部迁移完成
- src/utils/：12 个工具函数适配完成
- src/services/：15 个 service 文件迁移完成
- src/routes/：22 个路由适配完成

## 发现的问题
- Agent 1：发现 3 个 N+1 查询问题，待优化
- Agent 2：发现 2 个类型不匹配，需补充类型定义
- Agent 3：发现 1 个事务边界遗漏，需修复

## 验证结果
- ✅ 32/35 测试通过
- ⚠ 3 个测试失败（与迁移相关，待处理）

## 下一步行动
1. [TODO] 修复 3 个失败的测试
2. [TODO] 优化 N+1 查询
3. [TODO] 补充类型定义
4. [TODO] 生产环境数据库迁移评估
"
```

1.5.5 避坑指南：编辑 Agents 常见错误

错误一：过度并行

```
# × 错误：同时启动 10 个 Agent 修改 10 个文件，但这些文件都引用同一个类型
# 导致该类型文件被反复覆盖

# ✅ 正确做法：按依赖层级分组
第一波（4 个 Agent 并行）：处理各自独立的模块
第二波（1 个 Agent）：处理共享类型/常量
第三波（4 个 Agent 并行）：处理引用了共享类型的模块
```

原则： 并行度不是越高越好。控制在 3-5 个 Agent 同时运行效果最佳。

错误二：串行链条断裂

```
# × 错误：Agent 1 分析完直接启动 Agent 2，但 Agent 2 不知道 Agent 1 发现了什么
# Agent 2 重新分析了一遍，白白浪费了 Agent 1 的工作

# ✅ 正确做法：主对话必须作为"中转站"
Agent 1: "分析 src/auth/ ..."
```

```
主对话: "好的，已收到分析结果：发现 5 个问题，分别是..."
         "现在将结果传递给 Agent 2..."

Agent 2: "根据分析结果 [粘贴 Agent 1 的输出]，开始修复..."
```

错误三：不设置 maxTurns

```
# × 错误：Agent 处理大量文件时不设限制
Agent:
  instruction: "修改所有 src/ 下的文件，添加统一错误处理"

# ✅ 正确：根据任务量设置合理的 maxTurns
Agent:
  instruction: "修改 src/ 下的文件，添加统一错误处理"
  maxTurns: 25  # 估计：10 个文件 × 2-3 轮 = 25
```

错误四：忘记验证

```
# × 错误：Agent 改完所有文件就结束了，没有验证
# 结果：语法错误、类型错误全留到 CI 才发现

# ✅ 正确：每个阶段完成后必须有验证步骤
Agent:
  instruction: |
    迁移 src/services/ 到 Prisma：
    1. 修改每个 service 文件
    2. 修改后立即运行 npm run type-check
    3. 如果类型检查失败，立即修复
    4. 所有文件修改完成后，运行 npm test 验证功能
```

错误五：并行 Agent 之间的竞争状态

```
# × 错误：两个 Agent 同时修改一个有执行顺序要求的文件
Agent A: "在 src/app.ts 中添加 middleware A"
Agent B: "在 src/app.ts 中添加 middleware B（必须在 A 之后执行）"
# 如果 B 先完成，顺序就错了

# ✅ 正确做法：
# 方案 1：由一个 Agent 统一处理有顺序要求的部分
Agent: "统一处理 src/app.ts 的所有 middleware 配置"

# 方案 2：精确划分职责，Agent A 负责顺序无关的部分
Agent A: "只改 src/routes/ 下的路由文件"
Agent B: "在 src/app.ts 中添加所有 middleware（顺序敏感的统一处理）"
```

1.5.6 并行 vs 串行编辑 Agents：核心区别

一、写法上的区别

并行 — 所有 Agent 写在一起，同时发出：

```
# 单条消息里罗列多个 Agent，同时启动
Agent:
  instruction: "只修改 src/utils/ 下的文件"
  subagent_type: "general-purpose"
  description: "工具函数"

Agent:
  instruction: "只修改 src/components/ 下的文件"
  subagent_type: "general-purpose"
  description: "UI 组件"

Agent:
  instruction: "只修改 src/services/ 下的文件"
  subagent_type: "general-purpose"
  description: "业务逻辑"
```

串行 — 分多条消息发出，一个完成后才写下一个：

```
# ═══ 消息 1：发起第一个 Agent ═══════════════════════

Agent:
  instruction: "分析 src/，列出所有需要迁移的文件及依赖关系"
  subagent_type: "Explore"
  description: "分析阶段"
  maxTurns: 15

# 等待 Agent 返回结果后
# ═══ 消息 2：根据结果，发起第二个 Agent ═══════════

Agent:
  instruction: |
    基于分析结果，开始迁移：
    [将上一个 Agent 的输出粘贴在这里]

    迁移顺序：
    1. 先迁移共享工具函数（其他模块都依赖它）
    2. 再迁移独立的业务模块
    3. 最后迁移入口文件
    每次修改后运行 npm test 验证
  subagent_type: "general-purpose"
  description: "迁移阶段"
  maxTurns: 30
```

二、设计思路的区别

维度	并行 Agent	串行 Agent
任务关系	相互独立，不交叉	下一个依赖上一个的输出
文件范围	必须完全不重叠	可以有重叠，由主对话协调顺序
上下文传递	不需要传递	必须由主对话显式中转
失败影响	一个失败不影响其他	一个失败，后续全部暂停
总耗时	≈ 最慢那个 Agent 的耗时	= 所有 Agent 耗时之和
适用阶段	探索阶段、分析阶段、收尾验证	有依赖链的执行阶段

三、instruction 写法的区别

并行 Agent 的 instruction：强调**边界清晰**——只做自己的，不碰别人的：

```
✅ "只修改 src/utils/ 下的文件，不要改动 components/ 和 services/"
✅ "只处理用户模块，订单模块由另一个 Agent 负责"
✅ "只负责添加 TypeScript 类型，不改变任何逻辑"

× "处理 src/ 下的所有文件"          ← 并行时不能这样写，会争抢文件
× "同时修改 types/index.ts"          ← 共享文件不能并行处理
```

串行 Agent 的 instruction：强调**依赖关系**——我知道前面做了什么，我接着做：

```
✅ "基于刚才的分析结果 [粘贴分析输出]，继续迁移..."
✅ "上一步已创建了 prisma/schema.prisma，现在迁移 services 层"
✅ "前面 Agent 已经提取了共享类型，现在在各模块中应用这些类型"

× "重新扫描一遍项目"                ← 串行时不应重复前序工作
× "扫描需要迁移的文件"              ← 这是 Agent 1 的工作，Agent 2 不该重做
```

四、实操中的关键差异

```
┌──────────────────────────────────────────────────────────┐
│  并行：先规划，再一次性全部发出                            │
│                                                          │
│   你（规划者）：                                          │
│   "A 改 utils，B 改 components，C 改 services"            │
│   → 一口气把三个 Agent 全部写完，在同一条消息里发出去      │
```

```
│                                                                │
│   写的时候脑子里要有一张「文件边界地图」：                       │
│   Agent A 的笔不会碰到 Agent B 的文件                           │
└────────────────────────────────────────────────────────────────┘


┌────────────────────────────────────────────────────────────────┐
│  串行：跑一个，看结果，写下一个                                  │
│                                                                │
│   Agent 1 → 你看结果 → 你提炼关键信息 → 写给 Agent 2            │
│                                                                │
│   写 Agent 2 时必须做的三件事：                                 │
│   1. 知道 Agent 1 发现了什么 / 改动了什么                       │
│   2. 在 instruction 开头引用 Agent 1 的结果                     │
│   3. 明确告诉 Agent 2 不要重复 Agent 1 的扫描工作               │
└────────────────────────────────────────────────────────────────┘
```

五、什么时候选并行，什么时候选串行？

```
选并行：
  ├── 多个 Agent 改的是完全不同的目录
  ├── 多个 Agent 只是各自调研/分析，不写代码
  ├── 任务之间没有先后顺序要求
  └── 总任务量大，想节省时间

选串行：
  ├── 下一个要基于上一个的发现来行动
  ├── 下一个需要上一个创建的产物（文件/类型/配置）
  ├── 有严格的依赖顺序（先 schema → 再 service → 再 route）
  └── 不确定能不能并行，先探路再动
```

核心判断问题：

```
"Agent B 需要知道 Agent A 改了什么 / 发现了什么吗？"

  需要 ──────▶ 串行
  不需要 ─────▶ 并行
```

六、最常见的错误对照

错误	错误写法	正确写法
把串行误写成并行	A 分析、B 修复、C 测试写在一起同时发出	B 和 C 都依赖 A 的结果，必须等 A 完成后分开发

错误	错误写法	正确写法
把并行写成串行	Agent A 改 utils，等 A 完再让 B 改 components	B 不依赖 A，一口气全发出去节省时间
并行时不划边界	Agent A "处理 src/"，Agent B "处理 src/"	两个 Agent 都会扫到同一文件，必须明确边界
串行时不引用前序结果	Agent 2 "扫描需要迁移的文件"	Agent 2 应该基于 Agent 1 的结果继续，而不是重做
串行时链条断裂	Agent 1 → Agent 3（跳过了 Agent 2）	环环相扣，上一步是下一步的前提

```
# × 常见错误：串行链条断裂
Agent 1: "分析项目结构..."
Agent 3: "开始迁移..."  # 跳过了 Agent 2，直接用自己假设的前提
# Agent 3 可能基于错误假设工作，白费功夫

# ✅ 正确：每个串行 Agent 都显式引用前序结果
Agent 1: "分析项目结构..."
Agent 2: "基于 Agent 1 的分析 [粘贴结果]，开始逐个迁移..."
```

七、并行的最佳实践

1. **同批并行 Agent 数量控制在 3-5 个**：超过后管理复杂度急剧上升，且 Claude Code 上下文有限
2. **每个并行 Agent 必须有唯一的文件管辖范围**：用 `description` 字段标注清楚
3. **并行任务完成后，主对话必须汇总**：避免各 Agent 的改动各自为政
4. **并行适合探索 + 串行适合执行**：探索阶段并行调研，执行阶段串行迁移

```
# 并行探索阶段（不写代码，只分析）
Agent: "分析 src/utils/ 下的文件"     subagent_type: "Explore"
Agent: "分析 src/services/ 下的文件" subagent_type: "Explore"
Agent: "分析 src/components/"        subagent_type: "Explore"

# 串行执行阶段（基于探索结果，一步步迁移）
Agent: "基于探索结果，迁移 utils/ ..."   # 串行
Agent: "基于 utils 迁移结果，迁移 services/ ..."  # 串行
Agent: "基于 services 迁移结果，迁移 routes/ ..."  # 串行

# 并行收尾阶段（迁移全部完成后）
Agent: "扫描遗漏的硬编码"                subagent_type: "Explore"
Agent: "运行测试套件"                    subagent_type: "general-purpose"
```

1.6 Agent vs 主对话

特性	主对话	Agent

特性	主对话	Agent
上下文	共享当前项目的完整上下文	独立上下文，可选择性继承
并行性	单线程对话	可并行启动多个 Agent
任务专注度	多任务混合处理	单一任务专注处理
适用场景	日常对话、小任务、简单修改	复杂任务、并行处理、深度分析
资源消耗	单一上下文	每个 Agent 独立消耗
结果返回	即时	可选后台运行

选择建议：

```
日常任务 → 主对话
  └── 简单修改、问题咨询、文件操作

复杂任务 → Agent
  └── 大规模重构、深度分析、多方案研究

独立并行任务 → 多 Agent
  └── 同时调研多个主题、审查多个模块
```

1.7 最佳实践

实践一：明确的任务描述

```
# × 不够明确
"帮我看看代码"

# ✅ 明确具体
"审查 src/api/auth.ts，关注 JWT token 验证的安全性，
  检查是否正确处理 token 过期和刷新场景"
```

实践二：合理的 maxTurns

```
# × 过小，可能任务未完成
maxTurns: 3

# ✅ 根据任务复杂度设置
maxTurns: 20  # 复杂重构任务
```

实践三：并行任务分配

```
# 好的并行任务分配
Agent 1: "审查 src/api 的所有 endpoint"
Agent 2: "审查 src/utils 的所有工具函数"
Agent 3: "审查 src/components 的所有组件"

# 避免：任务重叠或相互依赖
```

实践四：后台任务监控

```
# 启动后台任务
Agent:
  instruction: "运行性能分析"
  run_in_background: true
  description: "性能分析"

# 等待完成后查看结果
```

第二章：Skill 技能系统

2.1 什么是 Skill

Skill 是预定义的命令或提示模板，通过简单的 `/命令` 语法快速执行特定任务。Skills 简化了常见工作流程，无需每次手动描述任务。

```
┌──────────────────────────────────────────┐
│            Skill 调用方式                  │
├──────────────────────────────────────────┤
│                                          │
│  /review          ──▶ 代码审查            │
│  /loop 5m ...     ──▶ 定时循环任务        │
│  /batch ...       ──▶ 批量并行任务        │
│  /simplify        ──▶ 代码简化审查        │
│                                          │
└──────────────────────────────────────────┘
```

2.2 Skill 完整列表

Skill	功能	用法
`/simplify`	审查变更代码，复用性/质量/效率检查	直接调用
`/loop`	定时循环执行命令或提示	`/loop 5m 任务描述`
`/batch`	大规模变更研究、规划和并行执行	`/batch 任务描述`
`/claude-api`	构建 Claude API 或 Anthropic SDK 应用	直接调用

Skill	功能	用法
`/init`	初始化新的 CLAUDE.md 项目文档	直接调用
`/review`	审查 Pull Request	`/review` 或 `@PR_URL`
`/security-review`	对当前分支变更进行安全审查	直接调用
`/pr-comments`	获取 GitHub Pull Request 的评论	`@PR_URL`
`/insights`	生成 Claude Code 会话分析报告	直接调用
`/statusline`	设置 Claude Code 状态栏 UI	直接调用

2.3 核心 Skill 详解

`/batch` - 批量并行任务

适用场景： 大规模代码迁移、重构、批量修改

基本用法：

```
# 迁移 React class 组件到 functional component
/batch 将所有 React class 组件迁移到 functional component

# 批量添加 TypeScript 类型
/batch 为所有 JavaScript 文件添加 TypeScript 类型定义

# 批量重构
/batch 将所有 callback 风格的 API 调用改写成 async/await
```

工作原理（完整流程）：

```
1. 研究代码库：将工作分解为 5-30 个独立单元
2. 呈现计划：展示每个单元的处理方案
3. 获得批准后：在隔离的 git worktree 中为每个单元生成后台 Agent
4. 每个 Agent 实现其单元、运行测试并打开 Pull Request
```

关键特性：

特性	说明
并行度	5-30 个并行 Agent（自动确定）
隔离	每个 Agent 在独立 git worktree 中运行，不影响当前分支
PR	每个单元完成后自动打开 PR
测试	每个 Agent 自动运行测试验证
适用条件	需要 git 仓库

> **注意：** `/batch` 会创建大量 git worktree。如果 Agent 未做任何更改，worktree 自动清理。大量 PR 可能会给代码审查带来压力，慎用于小规模任务。

`/loop` - 定时循环任务

适用场景： 监控、轮询、定时报告

基本用法：

```
# 每 5 分钟总结 AI 新闻
/loop 5m 总结最新的 AI 新闻

# 每 10 分钟检查错误日志
/loop 10m 检查错误日志并汇报异常

# 每小时汇报系统状态
/loop 1h 汇报系统资源使用情况
```

参数说明：

- 时间间隔：`数字 + 单位`
  - `m` = 分钟
  - `h` = 小时
- 任务描述：需要定期执行的任务

注意事项：

- 默认间隔：10 分钟
- 定时任务在 REPL 空闲时执行
- 会话结束后任务自动停止

`/review` - PR 审查

适用场景： 代码审查、PR 检查

基本用法：

```
# 审查当前分支的变更
/review

# 审查指定文件
/review src/components/UserProfile.tsx

# 审查远程 PR
/review @https://github.com/user/repo/pull/123
```

审查内容：

- ✅ 代码质量分析
- ✅ 潜在 Bug 检测
- ✅ 安全问题检查
- ✅ 性能优化建议
- ✅ 代码风格一致性
- ✅ 测试覆盖率

`/security-review` **- 安全审查**

适用场景： 安全审计、漏洞检测

```
/security-review
```

审查范围：

- ✕ SQL 注入风险
- ✕ XSS 漏洞
- ✕ 依赖安全问题
- ✕ 敏感信息暴露
- ✕ 认证授权问题
- ✕ CSRF 防护

`/simplify` **- 代码简化审查**

适用场景： 代码质量提升

```
# 全面审查
/simplify

# 聚焦特定问题
/simplify focus on memory efficiency
/simplify focus on error handling
```

审查内容：

- 📦 代码复用性分析
- 🔧 质量问题识别
- ⚡ 效率优化建议
- ✂ 简化建议

工作原理： 并行生成三个审查 Agent，汇总发现后应用修复。

`/debug` **- 调试模式**

适用场景： 诊断 Claude Code 异常行为、排查问题

```
# 启用调试并分析问题
/debug [描述问题]

# 启用调试（不描述问题）
/debug
```

工作原理：

1. 为当前会话启用详细日志记录（默认关闭）
2. 读取会话调试日志分析问题
3. 从 `/debug` 运行时从该时间点开始捕获日志

使用技巧：

- 在问题出现后运行 `/debug`，Claude 会读取已记录的日志
- 使用 `Ctrl+O` 切换详细模式查看完整日志
- 调试完成后会话继续运行，日志不再显示

`/insights` - 会话分析

适用场景： 了解 Claude Code 使用情况

```
/insights
```

输出内容：

- 📊 对话次数统计
- 💼 主要工作类型
- 🔧 使用最多的工具
- 💡 效率建议

`/init` - 初始化 CLAUDE.md

适用场景： 项目初始化

```
/init
```

自动引导创建项目的 `CLAUDE.md` 文档，包含：

- 项目概述
- 技术栈
- 代码规范
- 常用命令

`/claude-api` - Claude API 开发

适用场景： 构建 AI 应用

```
/claude-api
```

帮助构建使用 Claude API 的应用：

- API 集成
- SDK 使用
- 应用架构设计

2.4 自定义 Skill

自定义 Skill 使用 Markdown 文件定义（`.claude/commands/` 中的旧格式仍然兼容，但推荐使用新格式）。

文件结构

```
~/.claude/skills/                    # 个人 Skills（所有项目可用）
.claude/skills/                      # 项目级 Skills（仅当前项目可用）
├── my-skill/
│   ├── SKILL.md           # 主要说明（必需）
│   ├── reference.md       # 参考文档（可选）
│   ├── examples.md        # 示例输出（可选）
│   └── scripts/
│       └── helper.sh      # 辅助脚本（可选）
```

SKILL.md 格式

```
---
name: my-skill
description: 何时使用此 Skill，Claude 根据此字段决定是否自动调用
argument-hint: [参数提示]
disable-model-invocation: false
user-invocable: true
allowed-tools: Read, Grep
model: inherit
effort: medium
context: fork
agent: general-purpose
hooks: ...
---

你的 Skill 指令内容...
可以包含 Markdown 格式的完整说明。
```

Skill 位置与优先级

位置	路径	范围
企业级	托管设置	组织内所有用户
个人级	`~/.claude/skills/<skill>/SKILL.md`	所有项目
项目级	`.claude/skills/<skill>/SKILL.md`	仅当前项目
插件级	`<plugin>/skills/<skill>/SKILL.md`	启用插件时

> **优先级：** 企业 > 个人 > 项目。插件使用 `plugin-name:skill-name` 命名空间，不与其他级别冲突。

核心 frontmatter 字段

字段	默认值	说明
`name`	目录名	Skill 显示名称（小写字母、数字、连字符，最多 64 字符）
`description`	内容首段	功能描述，Claude 据此决定何时自动调用
`argument-hint`	—	自动补全时显示的参数提示，如 `[issue-number]`
`disable-model-invocation`	`false`	`true` = 禁止 Claude 自动调用（仅用户可手动调用）
`user-invocable`	`true`	`false` = 从 `/` 菜单隐藏（Claude 仍可调用）
`allowed-tools`	全部工具	Skill 活跃时无需授权即可使用的工具
`model`	inherit	此 Skill 使用的模型
`effort`	继承会话	努力级别：`low` / `medium` / `high` / `max`
`context`	—	`fork` = 在分叉 Subagent 中运行（隔离上下文）
`agent`	general-purpose	`context: fork` 时使用的 Subagent 类型
`hooks`	—	限定于此 Skill 生命周期的 Hooks

控制调用权限的两种模式

模式	你可以调用	Claude 可以调用	何时加载
默认	✅	✅	描述始终在上下文，调用时加载完整内容
`disable-model-invocation: true`	✅	×	你调用时才加载完整内容
`user-invocable: false`	×	✅	描述始终在上下文，Claude 主动使用

典型场景：

- `/deploy`：添加 `disable-model-invocation: true`，防止 Claude 主动部署
- 项目架构说明：添加 `user-invocable: false`，Claude 知道但用户看不到

字符串替换变量

变量	说明
`$ARGUMENTS`	调用时传递的所有参数
`$ARGUMENTS[N]`	按 0 基索引访问特定参数
`$N`	`$ARGUMENTS[N]` 的简写（`$0` = 第一个参数）
`${CLAUDE_SESSION_ID}`	当前会话 ID
`${CLAUDE_SKILL_DIR}`	SKILL.md 所在目录（用于引用捆绑的脚本）

示例：

```
---
name: migrate-component
description: 组件框架迁移
disable-model-invocation: true
---

迁移 $ARGUMENTS[0] 从 $ARGUMENTS[1] 到 $ARGUMENTS[2]。
保持所有现有行为和测试不变。
```

运行 `/migrate-component SearchBar React Vue`：

- `$ARGUMENTS[0]` = `SearchBar`
- `$ARGUMENTS[1]` = `React`
- `$ARGUMENTS[2]` = `Vue`

支持文件（参考文档、示例、脚本）

```
deploy-skill/
├── SKILL.md          # 主说明（必需，500 行以内）
├── reference.md      # 详细 API 参考（Claude 按需加载）
├── examples.md        # 用法和输出示例
└── scripts/
    ├── validate.sh    # Claude 可执行的验证脚本
    └── template.sh    # Claude 可执行的模板脚本
```

从 `SKILL.md` 中引用支持文件：

```
详细规范请参考 [reference.md](reference.md)
使用示例请参考 [examples.md](examples.md)
```

动态上下文注入（命令预执行）

使用 `!`<command>`` 语法，在 Skill 内容发送给 Claude **之前**运行命令，将输出插入到提示中：

```
---
name: pr-summary
description: 总结 Pull Request 的变更
context: fork
agent: Explore
allowed-tools: Bash(gh *)
---

## PR 上下文
- PR diff: !`gh pr diff`
- PR comments: !`gh pr view --comments`
- 变更文件: !`gh pr diff --name-only`

## 你的任务
基于以上信息总结这个 PR...
```

Skill 执行时，三个反引号命令先运行，输出替换占位符，Claude 收到带实际数据的提示。这是**预处理**，不是 Claude 执行的内容。

在 Subagent 中运行 Skill

使用 `context: fork` 让 Skill 在隔离的 Subagent 中运行：

```
---
name: deep-research
description: 深度研究主题
context: fork
agent: Explore
---

Research $ARGUMENTS thoroughly:

1. Find relevant files using Glob and Grep
2. Read and analyze the code
3. Summarize findings with specific file references
```

方向	系统提示	任务	CLAUDE.md
`context: fork` Skill	来自 agent 类型	SKILL.md 内容	✅ 加载

方向	系统提示	任务	CLAUDE.md
Subagent `skills` 字段	Subagent 正文	Claude 委派消息	✅ 加载

限制 Claude 的 Skill 访问

通过 `permissions` 设置控制 Claude 可以调用哪些 Skill：

```
// settings.json
{
  "permissions": {
    "deny": ["Skill(deploy *)"]   // 阻止所有以 deploy 开头的 Skill
  }
}
```

语法：`Skill(name)` = 精确匹配，`Skill(name *)` = 前缀匹配。

使用自定义 Skill

```
# 手动调用
/my-skill
/my-skill arg1 arg2

# Claude 根据 description 自动调用（如果未设置 disable-model-invocation）
```

使用自定义 Skill：

```
/format-check
/test-coverage
/deploy
```

实战案例一：CI/CD 质量门禁（前端团队场景）

背景： 一个 Next.js 项目，团队希望在每次 Claude Code 处理代码后，自动运行质量检查。

需求分析：

- 代码格式化检查（Prettier）
- ESLint 检查
- TypeScript 类型检查
- 自动化测试

配置实现：

```
// 项目 .claude/settings.json
{
  "skills": {
    "pre-commit": {
      "description": "提交前质量门禁检查",
      "command": "npm run lint && npm run type-check && npm test",
      "prompt": "执行提交前检查：运行 lint + 类型检查 + 单元测试\n如果任何一步失败，列出具体错误并给出修复建议\n如果全部通过，输出「✅ 质量门禁通过」"
    },
    "build-prod": {
      "description": "生产环境构建并检查",
      "command": "npm run build",
      "prompt": "执行生产构建：\n1. 运行 npm run build\n2. 如果构建成功，分析 bundle 大小变化\n3. 如果有警告，解释每个警告的影响\n4. 检查是否有大文件未做 code splitting"
    },
    "full-test": {
      "description": "运行完整测试套件",
      "command": "npm run test:coverage -- --coverageReporters=lcov",
      "prompt": "运行测试覆盖率检查：\n1. 执行完整测试套件\n2. 分析覆盖率报告，找出低于 80% 的模块\n3. 列出最需要补充测试的 5 个文件\n4. 检查是否有 flaky test（经常失败的测试）"
    }
  }
}
```

使用方式：

```
# 开发中随时检查代码质量
/pre-commit

# 提交前确认构建正常
/build-prod

# 冲刺阶段检查测试覆盖
/full-test
```

为什么用 Skill 而不是直接运行命令？

直接运行命令	自定义 Skill
只得到命令输出	命令输出 + AI 分析解读
需自己判断问题	AI 自动给出修复建议
命令结果需手动解读	智能汇总关键问题

实战案例二：数据库迁移助手（后端团队场景）

背景： 项目使用 Prisma ORM，有频繁的数据库迁移需求，团队希望 Claude Code 能智能辅助迁移工作。

配置实现：

```
// .claude/settings.json
{
  "skills": {
    "migrate-status": {
      "description": "检查数据库迁移状态",
      "command": "npx prisma migrate status",
      "prompt": "检查 Prisma 迁移状态：\n1. 分析当前迁移状态\n2. 列出未执行的迁移
\n3. 检查是否存在 drift（数据库实际结构与 schema 不同步）\n4. 给出迁移建议（是否安全
执行 pending 迁移）"
    },
    "migrate-dev": {
      "description": "开发环境数据库迁移",
      "command": "npx prisma migrate dev",
      "prompt": "在开发环境执行 Prisma 迁移：\n1. 运行 npx prisma migrate
dev\n2. 审查生成的 SQL 是否符合预期\n3. 如果有问题，询问是否要调整 schema 后重试\n4.
确认后应用迁移"
    },
    "migrate-prod": {
      "description": "生产环境安全迁移（只生成 SQL，不执行）",
      "command": "npx prisma migrate dev --create-only && npx prisma
migrate resolve --applied <LATEST>",
      "prompt": "生产环境迁移安全流程：\n⚠ 生产迁移必须谨慎！\n1. 先生成迁移文件：
npx prisma migrate dev --create-only\n2. 检查生成的 SQL 是否有破坏性操作（DROP
COLUMN 等）\n3. 提供两个选项：\n   A) 确认安全，执行迁移\n   B) 需要调整，修改
schema 后重新生成\n4. 强调数据备份的重要性"
    },
    "db-pull": {
      "description": "从生产数据库拉取 schema 到本地",
      "command": "npx prisma db pull",
      "prompt": "从数据库反向生成 Prisma schema：\n1. 运行 db pull 同步最新结构
\n2. 检查是否有字段类型变更需要注意\n3. 列出 schema 变更摘要"
    }
  }
}
```

典型工作流：

```
# 开发新功能前检查迁移状态
/migrate-status

# 修改 schema 后，在开发环境测试迁移
/migrate-dev

# 功能完成后，准备生产迁移
/migrate-prod
```

实战案例三：Monorepo 多包管理（大型项目场景）

背景： 项目使用 PNPM Workspace，包含 packages/core、packages/api、packages/web 三个子包。

配置实现：

```
{
  "skills": {
    "check-all": {
      "description": "检查所有包的 lint + 类型 + 测试",
      "command": "pnpm -r run lint && pnpm -r run type-check && pnpm -r run test",
      "prompt": "检查 monorepo 所有包的代码质量：\n依次对 packages/core、packages/api、packages/web 执行：\n1. ESLint 检查\n2. TypeScript 类型检查\n3. 单元测试\n按包输出检查结果，汇总失败情况"
    },
    "build-all": {
      "description": "按依赖顺序构建所有包",
      "command": "pnpm -r run build",
      "prompt": "构建 monorepo 所有包（已按 topological 顺序）：\n1. 先构建 packages/core\n2. 再构建 packages/api\n3. 最后构建 packages/web\n分析每个包的构建时间和产物大小"
    },
    "bump-core": {
      "description": "升级 core 包版本并同步依赖",
      "command": "cd packages/core && npm version patch && cd ../.. && pnpm install",
      "prompt": "升级 core 包版本：\n1. 确认当前 core 版本\n2. 选择版本类型：patch / minor / major\n3. 执行版本升级\n4. 更新所有依赖 core 的包的 package.json\n5. 确认 workspace 协议（workspace:*）是否正确"
    }
  }
}
```

实战案例四：无命令纯提示型 Skill

背景： 有些任务不需要运行具体命令，而是需要 Claude 以特定角色/流程来处理。

```
{
  "skills": {
    "explain-code": {
      "description": "用中文详细解释代码逻辑",
      "prompt": "你是一位资深技术导师。请详细解释以下代码：\n1. 整体功能和设计思路\n2. 关键函数的作用（逐个说明）\n3. 数据流向和处理流程\n4. 可能的边界情况和错误处理\n5. 如果有更好的写法，给出建议\n请用清晰的中文输出，适合初级到中级工程师理解"
    },
    "review-pr": {
      "description": "代码审查（结构化输出）",
```

```
      "prompt": "作为资深代码审查者，请对代码变更进行全面审查，\n按以下结构输出：\n\n## 变更概述\n简要描述这次改动的目的\n\n## 优点\n列出代码中做得好的地方\n\n## 需要改进\n列出具体问题，每条附上：\n- 问题描述\n- 风险等级（高/中/低）\n- 修复建议\n\n## 安全性\n检查：注入风险、认证授权、敏感信息泄露\n\n## 性能\n检查：N+1 查询、不必要的重复计算、大文件处理\n\n## 建议提问\n列出 2-3 个需要向作者确认的问题"
    },
    "onboard": {
      "description": "新成员项目上手引导",
      "prompt": "作为团队技术负责人，请为新加入的开发者提供上手引导：\n1. 项目整体架构（一句话说清楚这是什么项目）\n2. 技术栈概览\n3. 本地开发环境搭建步骤\n4. 如何启动项目并运行第一个功能\n5. 代码规范要点（Naming/Import 顺序/提交规范）\n6. 遇到问题时的求助渠道\n请用友好的语气，假设对方有 2 年开发经验但不了解本项目"
    }
  }
}
```

使用方式：

```
# 审查任何文件时调用
/review-pr

# 新人入职时使用
/onboard

# 学习陌生代码时使用
/explain-code
```

纯提示型 Skill 的优势： 标准化输出格式，确保每次审查的维度和深度一致，新人也能快速上手。

实战案例五：组合型 Skill（命令 + 智能分析）

背景： 部署前需要完整检查，但不同环境的检查重点不同。

```
{
  "skills": {
    "deploy-staging": {
      "description": "部署到预发布环境前检查",
      "command": "npm run build && npm run test:e2e",
      "prompt": "执行预发布环境部署前检查：\n1. 运行生产构建，确认无错误\n2. 运行 E2E 测试，确认核心流程正常\n3. 如果检查通过，输出：「预发布环境已准备就绪」\n4. 如果有失败项，列出具体失败用例及原因\n此技能不自动执行部署，需要手动确认"
    },
    "hotfix-check": {
      "description": "热修复专用快速检查",
      "command": "npm run lint -- --max-warnings 0 && npm test -- --testPathIgnorePatterns=e2e",
      "prompt": "热修复场景的快速质量检查：\n与完整检查不同，热修复要求速度和最小化影响：\n1. 只运行 lint + 单元测试，跳过 E2E\n2. 检查改动的文件是否触发了不该有的依赖变
```

```
化\n3. 确认改动范围最小化（diff 不应超过 5 个文件）\n4. 如果通过，提示可以直接提
PR\n5. 如果范围过大，警告可能不适合热修复"
      }
    }
  }
```

2.5 Skill 场景对照表

场景	推荐 Skill	说明
大规模代码迁移	`/batch`	并行处理大量文件
定时监控任务	`/loop`	持续监控和报告
代码审查	`/review`	质量检查和反馈
安全审计	`/security-review`	漏洞检测
API 开发	`/claude-api`	AI 应用构建
项目初始化	`/init`	创建项目文档
会话分析	`/insights`	使用统计
代码质量	`/simplify`	简化建议

2.6 Hooks 系统详解

什么是 Hooks

Hooks 是在 Claude Code 生命周期中的特定时刻自动执行的 shell 命令。与 Skill（基于提示）不同，Hook 是**确定性**的——指定条件触发时必定执行，不依赖 LLM 判断。

```
┌──────────────────────────────────────────────────────────┐
│                    Claude Code 生命周期                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  SessionStart ──▶ UserPromptSubmit ──▶ PreToolUse    │
│        │                                          │    │
│    会话开始时              提交提示时              工具执行前 │
│                                                          │
│  PostToolUse ──▶ Stop ──▶ SessionEnd                 │
│      │                                        │    │
│  工具执行后            响应结束时              会话结束时 │
│                                                          │
│  每个节点都可以挂载 Hook，自动执行特定操作            │
└──────────────────────────────────────────────────────────┘
```

Hook 配置位置与范围

位置	范围	可共享
`~/.claude/settings.json`	所有项目（个人）	否
`.claude/settings.json`	当前项目	是，可提交到仓库
`.claude/settings.local.json`	当前项目	否，gitignored
托管策略设置	组织范围	是，管理员控制
Plugin `hooks/hooks.json`	启用插件时	是
Skill/Agent frontmatter	Skill/Agent 活跃时	是

> 运行 `/hooks` 可查看所有已配置的 Hook（只读菜单，需修改请直接编辑 JSON）。

完整事件列表

事件	触发时机	支持匹配器
`SessionStart`	会话开始或恢复时	会话启动方式（startup/resume/clear/compact）
`UserPromptSubmit`	你提交提示后、Claude 处理前	不支持
`PreToolUse`	工具执行**前**	工具名称（如 `Bash`、`Edit|Write`）
`PermissionRequest`	权限对话框出现时	工具名称
`PostToolUse`	工具执行**成功后**	工具名称
`PostToolUseFailure`	工具执行**失败后**	工具名称
`Notification`	Claude Code 发送通知时	通知类型
`SubagentStart`	Subagent 开始执行时	代理类型
`SubagentStop`	Subagent 完成后	代理类型
`Stop`	Claude 响应完成时	不支持
`StopFailure`	API 错误导致回合结束时	错误类型
`PreCompact`	上下文压缩前	触发原因
`PostCompact`	上下文压缩完成后	触发原因
`InstructionsLoaded`	CLAUDE.md 或规则文件加载时	加载原因
`ConfigChange`	配置文件在会话中变更时	配置源
`CwdChanged`	工作目录变更时	不支持
`FileChanged`	监视的文件变更时	文件名
`WorktreeCreate`	git worktree 创建时	不支持

事件	触发时机	支持匹配器
`WorktreeRemove`	git worktree 删除时	不支持
`SessionEnd`	会话终止时	结束原因

核心事件详解

PreToolUse — 执行前拦截

最强大的 Hook 之一。可验证、阻止或修改工具调用。

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "./scripts/validate-command.sh"
          }
        ]
      }
    ]
  }
}
```

PostToolUse — 执行后自动处理

在文件编辑后自动格式化：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | xargs npx prettier
--write"
          }
        ]
      }
    ]
  }
}
```

SessionStart — 恢复上下文

压缩后重新注入关键信息：

```
{
  "hooks": {
    "SessionStart": [
      {
        "matcher": "compact",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Reminder: 使用 Bun 而非 npm，提交前运行 bun test'"
          }
        ]
      }
    ]
  }
}
```

Hook 与 Claude Code 的通信方式

方式一：退出码（简单控制）

退出码	含义
0	操作继续（stdout 内容加入上下文）
2	阻止操作（stderr 变为 Claude 的反馈）
其他	操作继续，stderr 仅记录

```bash
#!/bin/bash
# 阻止对 .env 文件的编辑
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

if [[ "$FILE_PATH" == *".env"* ]]; then
  echo "Blocked: .env files cannot be edited directly" >&2
  exit 2
fi

exit 0
```

方式二：结构化 JSON 输出（精细控制）

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Use rg instead of grep"
  }
}
```

不同的 Hook 事件支持不同的决策字段：

事件	允许的决策
`PreToolUse`	`allow` / `deny` / `ask`
`PostToolUse` / `Stop`	`block`
`PermissionRequest`	`behavior: allow`
`SessionStart` / `UserPromptSubmit`	`additionalContext`

匹配器语法

事件类型	匹配字段	示例
`PreToolUse` 等	工具名称	`Bash`、`Edit|Write`、`mcp__github__.*`
`SessionStart`	启动方式	`startup`、`resume`、`compact`
`SessionEnd`	结束原因	`clear`、`logout`、`bypass_permissions_disabled`
`SubagentStart/Stop`	代理类型	`Explore`、`Plan`、自定义名称
`FileChanged`	文件名	`.envrc`、`.env`
`ConfigChange`	配置源	`user_settings`、`skills`、`project_settings`

Hook 类型详解

除了 `type: command`，还有三种类型：

类型	说明	适用场景
`command`	运行 shell 命令（最常用）	格式化、验证、拦截
`prompt`	单轮 LLM 评估，返回 `{"ok": true/false}`	需要判断的规则
`agent`	多轮 Subagent 验证（60s 超时，50 轮）	需要读取文件/运行命令的验证
`http`	POST JSON 到 URL	与外部服务集成

```
// 基于提示的 Hook：让模型判断是否可以停止
{
  "hooks": {
```

```
    "Stop": [
      {
        "hooks": [
          {
            "type": "prompt",
            "prompt": "检查所有任务是否完成。如果未完成，返回 {\"ok\": false,
\"reason\": \"...\"}"
          }
        ]
      }
    ]
  }
}
```

```
// HTTP Hook：将工具调用 POST 到日志服务
{
  "hooks": {
    "PostToolUse": [
      {
        "hooks": [
          {
            "type": "http",
            "url": "http://localhost:8080/hooks/tool-use",
            "headers": { "Authorization": "Bearer $MY_TOKEN" },
            "allowedEnvVars": ["MY_TOKEN"]
          }
        ]
      }
    ]
  }
}
```

实战案例：保护敏感文件

目标： 阻止 Claude 修改 `.env`、`package-lock.json` 和 `.git/` 中的文件。

```
# 1. 创建保护脚本
mkdir -p .claude/hooks
cat > .claude/hooks/protect-files.sh << 'EOF'
#!/bin/bash
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

PROTECTED_PATTERNS=(".env" "package-lock.json" ".git/")

for pattern in "${PROTECTED_PATTERNS[@]}"; do
  if [[ "$FILE_PATH" == *"$pattern"* ]]; then
    echo "Blocked: $FILE_PATH matches protected pattern '$pattern'" >&2
    exit 2
```

```
  fi
done

exit 0
EOF
chmod +x .claude/hooks/protect-files.sh

# 2. 注册 Hook
```

在 `.claude/settings.json` 中注册：

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"
          }
        ]
      }
    ]
  }
}
```

实战案例：Subagent 生命周期 Hooks

在 `settings.json` 中配置，响应 Subagent 启动/停止事件：

```
{
  "hooks": {
    "SubagentStart": [
      {
        "matcher": "db-agent",
        "hooks": [
          { "type": "command", "command": "./scripts/setup-db.sh" }
        ]
      }
    ],
    "SubagentStop": [
      {
        "hooks": [
          { "type": "command", "command": "./scripts/cleanup-db.sh" }
        ]
      }
    ]
```

```
    }
  }
```

常见问题排查

问题	原因	解决
Hook 不触发	匹配器拼写错误	检查 `Ctrl+O` 详细模式输出
脚本无执行权限	未 `chmod +x`	必须使脚本可执行
JSON 解析失败	shell 配置文件有 echo	在 `.bashrc`/`zshrc` 中加 `[[ $- == *i* ]]` 保护
Stop Hook 死循环	未处理 `stop_hook_active`	检查 `stop_hook_active: true` 时提前 exit 0

2.7 MCP 系统详解

什么是 MCP

MCP（Model Context Protocol）是一个开源标准，通过标准化接口将 Claude Code 连接到外部工具、数据源和 API。连接后，你可以用自然语言驱动这些工具。

```
┌──────────────────────────────────────────────────────────┐
│                    Claude Code                           │
│                                                          │
│  "查询 Sentry 过去 24 小时最常见的错误"                  │
└──────────────────────┬───────────────────────────────────┘
                       │ MCP 协议
              ┌────────▼────────┐
              │  MCP Server     │
              │  (Sentry)       │
              └─────────────────┘
                       │
              ┌────────▼────────┐
              │  外部服务       │
              │  (Sentry)       │
              └─────────────────┘
```

MCP 可以做什么

能力	示例
从 Issue 跟踪器实现功能	"添加 JIRA ENG-4521 中描述的功能并创建 PR"
分析监控数据	"检查 Sentry 和 Statsig 中 ENG-4521 功能的使用情况"
查询数据库	"查找 PostgreSQL 中使用某功能的 10 个随机用户"
集成设计	"根据 Slack 中新发布的 Figma 设计更新邮件模板"
自动化工作流	"创建 Gmail 草稿，邀请 10 个用户参加功能反馈会议"

三种传输类型

类型	说明	适用场景
HTTP（推荐）	连接到远程 MCP 服务器	云服务、最广泛支持
SSE	Server-Sent Events	旧版传输，已弃用
stdio	本地进程通信	需要本地系统访问的工具

添加 HTTP 服务器：

```
# 基本
claude mcp add --transport http <name> <url>

# 真实示例：Notion
claude mcp add --transport http notion https://mcp.notion.com/mcp

# 带认证
claude mcp add --transport http github https://api.githubcopilot.com/mcp/
```

添加本地 stdio 服务器：

```
# 基本语法
claude mcp add --transport stdio --env API_KEY=YOUR_KEY <name> -- <command>
[args...]

# 真实示例：Airtable
claude mcp add --transport stdio --env AIRTABLE_API_KEY=YOUR_KEY airtable \
  -- npx -y airtable-mcp-server

# Windows 需要 cmd /c 包装
claude mcp add --transport stdio my-server -- cmd /c npx -y @some/package
```

> **重要：** 所有选项（`--transport`、`--env`、`--scope`）必须在服务器名称**之前**，`--` 分隔服务器名称和传递给 MCP 服务器的参数。

安装范围与配置位置

范围	说明	存储位置
local（默认）	仅当前项目对你可用	`~/.claude.json`
project	通过 `.mcp.json` 共享给团队，可版本控制	项目根目录 `.mcp.json`
user	在你所有项目中可用	`~/.claude.json`

```
# 添加项目范围服务器（团队共享）
claude mcp add --transport http paypal --scope project
```

```
https://mcp.paypal.com/mcp

# 添加用户范围服务器
claude mcp add --transport http hubspot --scope user
https://mcp.hubspot.com/anthropic
```

项目范围 `.mcp.json` 示例：

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "${GITHUB_MCP_URL}",
      "headers": {
        "Authorization": "Bearer ${GITHUB_TOKEN}"
      }
    }
  }
}
```

环境变量扩展：`${VAR}` 或 `${VAR:-default}`。

MCP 管理命令

```
claude mcp list           # 列出所有配置的服务器
claude mcp get <name>     # 查看特定服务器详情
claude mcp remove <name> # 删除服务器
claude mcp reset-project-choices  # 重置项目范围服务器的批准选择
/mcp                      # 在 Claude Code 中检查服务器状态
```

MCP 资源引用

MCP 服务器可以暴露资源，用 `@` 引用（类似文件引用）：

```
Can you analyze @github:issue://123 and suggest a fix?
Compare @postgres:schema://users with @docs:file://database/user-model
```

MCP 提示命令

MCP 服务器可以提供提示，在 Claude Code 中作为命令使用：

```
/mcp__github__list_prs
/mcp__github__pr_review 456
/mcp__jira__create_issue "Bug in login flow" high
```

MCP 认证

OAuth 2.0（需要浏览器）：

```
# 1. 添加服务器
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp

# 2. 在 Claude Code 中认证
/mcp
# 按提示在浏览器中登录
```

预配置 OAuth 凭据（CI/自动化）：

```
# 使用客户端 ID 和密钥
claude mcp add --transport http \
  --client-id your-client-id --client-secret --callback-port 8080 \
  my-server https://mcp.example.com/mcp

# 通过环境变量（CI 场景）
MCP_CLIENT_SECRET=your-secret claude mcp add --transport http \
  --client-id your-client-id --client-secret --callback-port 8080 \
  my-server https://mcp.example.com/mcp
```

MCP 工具搜索（按需加载）

当配置的 MCP 服务器很多时，工具定义会消耗大量上下文。MCP 工具搜索自动延迟加载，只在需要时发现工具。

```
触发条件：MCP 工具描述超过上下文窗口的 10%
行为：工具被延迟加载，Claude 用搜索工具按需发现
```

控制方式：

```
# 使用 5% 阈值
ENABLE_TOOL_SEARCH=auto:5 claude

# 完全禁用
ENABLE_TOOL_SEARCH=false claude
```

企业托管 MCP 配置

方式一：`managed-mcp.json` 独占控制

IT 管理员在系统目录部署固定服务器集，用户无法添加：

```
macOS:   /Library/Application Support/ClaudeCode/managed-mcp.json
Linux:   /etc/claude-code/managed-mcp.json
Windows: C:\Program Files\ClaudeCode\managed-mcp.json
```

方式二：允许列表/拒绝列表

```
{
  "allowedMcpServers": [
    { "serverName": "github" },
    { "serverUrl": "https://mcp.company.com/*" },
    { "serverCommand": ["npx", "-y", "approved-package"] }
  ],
  "deniedMcpServers": [
    { "serverName": "dangerous-server" }
  ]
}
```

常用 MCP 服务器推荐

服务器	用途	命令
GitHub	PR 审查、Issue 管理	`claude mcp add --transport http github https://api.githubcopilot.com/mcp/`
Sentry	错误监控分析	`claude mcp add --transport http sentry https://mcp.sentry.dev/mcp`
PostgreSQL	数据库查询	`claude mcp add --transport stdio db -- npx -y @bytebase/dbhub --dsn "postgresql://..."`

2.8 Plugins 插件系统详解

什么是 Plugins

Plugins 是将 Skills、Agents、Hooks 和 MCP 服务器打包成分发单元的方式。与独立配置相比，Plugins 支持版本控制、团队共享和市场分发。

```
Plugins 与独立配置对比：

独立配置（.claude/）
├── 优点：快速实验，简短名称 /hello
└── 缺点：只能手动复制分享

Plugins
```

```
├── 优点：版本化、市场分发、团队共享
└── 缺点：Skills 命名空间化（/my-plugin:hello）
```

何时用 Plugins vs 独立配置

场景	推荐
单项目个人工作流	独立配置
多项目共享	Plugins（user 范围）
团队协作	Plugins（项目/市场）
快速实验	独立配置，成熟后迁移到 Plugins

插件目录结构

```
my-plugin/
├── .claude-plugin/
│   └── plugin.json          # 插件清单（必需）
├── commands/                 # Markdown 命令（Skills 的旧格式）
├── skills/                   # Agent Skills（SKILL.md 格式）
│   └── code-review/
│       └── SKILL.md
├── agents/                   # 自定义 Agent 定义
├── hooks/
│   └── hooks.json            # Hooks 配置
├── .mcp.json                # MCP 服务器配置
├── .lsp.json                # LSP 服务器配置
├── settings.json             # 启用时的默认设置
└── README.md                # 使用文档
```

> **注意：** `commands/`、`agents/`、`skills/`、`hooks/` 必须在插件**根目录**，不能在 `.claude-plugin/` 内。

plugin.json 清单

```json
{
  "name": "code-quality-tools",
  "description": "代码质量工具集：审查、Lint、安全检查",
  "version": "1.0.0",
  "author": {
    "name": "Your Name",
    "email": "you@example.com"
  },
  "homepage": "https://github.com/you/plugin",
  "repository": "https://github.com/you/plugin",
  "license": "MIT"
}
```

开发与测试

本地测试（开发期间）：

```
# 加载本地插件
claude --plugin-dir ./my-plugin

# 加载多个插件
claude --plugin-dir ./plugin-one --plugin-dir ./plugin-two

# 热重载（无需重启）
/reload-plugins
```

测试检查清单：

- `/my-plugin:skill-name` — Skill 是否可用
- `/agents` — Agent 是否出现
- 触发条件 — Hook 是否按预期执行

创建你的第一个插件

```
# 1. 创建目录结构
mkdir -p my-first-plugin/.claude-plugin
mkdir -p my-first-plugin/skills/hello

# 2. 创建清单
cat > my-first-plugin/.claude-plugin/plugin.json << 'EOF'
{
  "name": "my-first-plugin",
  "description": "一个学习插件基础的问候插件",
  "version": "1.0.0"
}
EOF

# 3. 创建 Skill
cat > my-first-plugin/skills/hello/SKILL.md << 'EOF'
---
name: hello
description: 友好地问候用户
---

Greet the user warmly and ask how you can help them today.
EOF

# 4. 测试
claude --plugin-dir ./my-first-plugin
# 使用 /my-first-plugin:hello
```

插件 Skill 的命名空间

插件中的 Skill 以插件名为前缀：

- 插件名 `my-first-plugin` + Skill 名 `hello` → `/my-first-plugin:hello`

将独立配置迁移到插件

```
# 1. 创建插件结构
mkdir -p my-plugin/.claude-plugin

# 2. 创建清单
cat > my-plugin/.claude-plugin/plugin.json << 'EOF'
{
  "name": "my-plugin",
  "description": "从独立配置迁移",
  "version": "1.0.0"
}
EOF

# 3. 复制现有配置
cp -r .claude/commands my-plugin/
cp -r .claude/agents my-plugin/
cp -r .claude/skills my-plugin/

# 4. 迁移 Hooks（从 settings.json 复制 hooks 对象）
cat > my-plugin/hooks/hooks.json << 'EOF'
{
  "hooks": {
    "PostToolUse": [...]
  }
}
EOF

# 5. 测试
claude --plugin-dir ./my-plugin
```

插件分发

1. **添加 README.md** — 包含安装和使用说明
2. **语义版本控制** — 在 `plugin.json` 中维护版本
3. **创建/使用市场** — 通过插件市场分发
4. **提交到官方市场**：
   - claude.ai
   - platform.claude.com

插件默认设置

在 `settings.json` 中设置，启用插件时自动应用：

```
{
  "agent": "security-reviewer"
}
```

激活插件时，自动将 `security-reviewer` Agent 设为主会话 Agent。

第三章：斜杠命令与交互

3.1 斜杠命令完整列表

命令	功能	参数
`/help`	显示帮助信息	无
`/clear`	清除对话历史	无
`/compact`	压缩上下文，保留关键信息	无
`/model`	切换使用的模型	`opus`/`sonnet`/`haiku`
`/exit` 或 `/quit`	退出 Claude Code	无
`/feedback`	提交功能请求或 Bug 报告	无
`/review`	代码审查	文件路径或 PR URL
`/security-review`	安全审查	无
`/debug`	启用调试模式	无
`/simplify`	简化代码审查	无
`/batch`	批量并行任务	任务描述
`/loop`	定时循环任务	时间间隔 + 任务
`/init`	初始化 CLAUDE.md	无
`/insights`	会话分析	无
`/pr-comments`	获取 PR 评论	PR URL
`/statusline`	状态栏设置	配置选项

3.2 核心命令详解

`/model` - 模型切换

```
/model opus       # 最强模型，适合复杂分析
/model sonnet     # 平衡模型，推荐日常使用
/model haiku      # 快速模型，适合简单任务
```

切换时机：

- 开始复杂任务时 → 切换到 Opus
- 日常开发 → 使用 Sonnet
- 简单查询 → 使用 Haiku

`/clear` - 清除对话

```
/clear
```

效果：

- ✅ 清除所有对话历史
- ✅ 保留项目上下文
- ✅ 保留文件内容
- × 不改变已打开的文件

使用场景：

- 对话变得混乱时
- 开始新话题时
- 上下文太长影响性能时

`/compact` - 上下文压缩

```
/compact
```

效果：

- 压缩对话历史
- 保留关键信息（决策、结论、重要上下文）
- 释放上下文空间

使用场景：

- 上下文快满时
- 长任务中途需要压缩

`/debug` - 调试模式

```
/debug
```

效果：

- ✅ 启用详细日志输出

- ✅ 显示工具调用详情
- ✅ 显示思考过程
- ✅ 帮助诊断问题

使用场景：

- Claude Code 行为异常时
- 排查问题原因
- 学习工具工作原理

3.3 快捷键

快捷键	功能
`Ctrl+C`	中断当前操作
`Ctrl+C` (两次)	退出 Claude Code
`Tab`	自动补全命令
`↑` / `↓`	浏览命令历史
`Ctrl+L`	清屏

3.4 对话技巧

提供上下文的技巧

技巧	示例	效果
指定文件路径	"在 src/utils/helpers.ts 中..."	精确定位
说明技术栈	"这是一个 Next.js 14 项目..."	正确理解
描述约束	"使用 TypeScript strict 模式..."	符合要求
提供示例	"示例输入... 输出..."	减少歧义

分步骤执行

对于复杂任务，分步骤效果更好：

```
1. "首先了解项目结构，列出目录树"
2. "分析 src/api/user.ts 的主要功能"
3. "基于以上分析，列出需要修改的文件"
4. "从修改配置文件开始执行"
```

3.5 有效的提示词

好的提示词特征

```
✅ 具体明确
✅ 包含上下文
✅ 有明确的期望
✅ 提供约束条件
```

示例对比

✗ 不推荐	✅ 推荐
"帮我看看"	"检查 src/api/user.ts 中的错误处理逻辑"
"修复它"	"修复 src/utils/format.ts 第 23 行的类型错误"
"写代码"	"用 TypeScript 写一个深拷贝函数，处理循环引用"
"创建组件"	"创建一个 React 组件，显示用户头像，使用 Tailwind CSS"

第四章：工具详解

4.1 Read - 文件读取

语法：

```
Read:
  file_path: "/path/to/file"
  limit: 100       # 可选，限制行数
  offset: 0        # 可选，起始行号
```

示例：

```
# 读取整个文件
"读取 src/utils/helpers.ts"

# 限制行数
"读取 src/App.tsx 的前 100 行"

# 跳过开头
"读取 src/main.ts 从第 50 行开始"

# 指定范围
"读取 src/config.ts 第 10-30 行"
```

4.2 Edit - 文件编辑

语法：

```
Edit:
  file_path: "/path/to/file"
  old_string: "要替换的原文"
  new_string: "替换后的内容"
  replace_all: false  # 是否全部替换
```

编辑技巧：

技巧一：提供足够上下文

```
# ✗ 可能匹配多处
old_string: "return result"

# ✅ 唯一匹配
old_string: "function calculateSum(arr) {\n  const result =
arr.reduce(...)\n  return result\n}"
```

技巧二：注意缩进和换行

```
# 包含完整缩进
old_string: "  const name = 'test'\n  console.log(name)"

# 使用 \n 表示换行
```

技巧三：批量替换

```
# 单次替换
Edit: old_string="var " new_string="let " replace_all=false

# 全部替换
Edit: old_string="console.log" new_string="logger.info" replace_all=true
```

4.3 Write - 文件写入

语法：

```
Write:
  file_path: "/path/to/file"
  content: "文件内容"
```

注意事项：

> ⚠ Write 会完全覆盖文件！只想修改部分请用 Edit。

```
# 创建新文件
"创建一个 React 组件 src/components/Button.tsx"

# 覆盖文件
"创建一个新的 package.json"
```

4.4 Glob - 文件搜索

语法：

```
Glob:
  pattern: "**/*.ts"
  path: "/path/to/dir"  # 可选
```

常用模式：

模式	含义
`*.ts`	根目录的 .ts 文件
`**/*.ts`	所有 .ts 文件（递归）
`src/**/*`	src 目录下所有文件
`**/index.*`	所有名为 index 的文件

4.5 Grep - 内容搜索

语法：

```
Grep:
  path: "/path/to/dir"
  pattern: "正则表达式"
  output_mode: "content"  # content | files_with_matches | count
  context: 3              # 显示上下文行数
```

输出模式：

模式	说明
`content`	显示匹配行内容和行号
`files_with_matches`	只返回文件名
`count`	返回匹配次数

4.6 Bash - 终端执行

语法：

```
Bash:
  command: "npm install"
  description: "安装项目依赖"
  timeout: 120000  # 超时时间（毫秒）
```

常用场景：

```
# 安装依赖
"npm install"

# 运行脚本
"npm run build"

# 查看输出
"node server.js"

# 组合命令
"npm install && npm run build"
```

实践练习

练习一：Agent 并行任务

目标： 体验 Agent 的并行处理能力

任务：

1. 启动 3 个 Agent 同时审查项目的不同模块
2. 比较并行 vs 串行的效率差异

练习二：Skill 技能使用

目标： 掌握常用 Skill

任务清单：

1. ✅ 使用 `/review` 审查一个代码文件
2. ✅ 使用 `/loop 1m` 设置一个简短定时任务
3. ✅ 使用 `/insights` 查看会话分析
4. ✅ 尝试 `/simplify` 审查代码

练习三：斜杠命令

目标： 熟练使用斜杠命令

任务清单：

1. ✅ 使用 `/model` 切换不同模型
2. ✅ 使用 `/debug` 启用调试模式
3. ✅ 使用 `/clear` 清除对话
4. ✅ 使用 `/compact` 压缩上下文

本教程是 Claude Code 系统学习系列的第二部分。